Software Engineering Project Report

AI Resume Analyzer Platform

Submitted By

ID	Name
23103380	Ehetisum Sharif
23103388	Nishat Tasnim
23103399	Md. Sajid

Course No: CSC 470
Course Title: Software Engineering Lab
Section: D
Semester: Summer 2026
Date of submission: 02/09/2026

Submitted To

Avijit Biswas
Lecturer, Department of CSE

IUBAT - International University of Business Agriculture and Technology

IUBAT School of Computer Science and Engineering (ISCSE)
Department of Computer Science and Engineering

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Introduction

The AI Resume Analyzer Platform is a web application that helps a job seeker improve a resume before applying for a job. A user uploads a resume, pastes the description of the post they are applying for, and the system returns an ATS match score between 0% and 100%, written feedback grouped into four areas, and a list of the important keywords that are missing from the resume.

The idea comes from a real problem. Most online job applications are filtered by an Applicant Tracking System before a human ever reads them, and a rejected applicant is almost never told why. Students and fresh graduates therefore keep sending the same resume without knowing what is wrong with it. This project gives them that missing feedback, free of charge and within a few seconds.

The system is built as a layered application with a separate client. The backend is an ASP.NET Core 8 Web API written in C#, secured with ASP.NET Core Identity and JSON Web Tokens, and it stores data in SQL Server through Entity Framework Core 8. The frontend is a React 19 application built with Vite, styled with Tailwind CSS and using Zustand for state and Axios for API calls. The analysis itself is performed by Llama 3, an open source model from Meta, which runs locally through Ollama and is reached from the API through Microsoft Semantic Kernel. Because the model is local, no resume ever leaves the machine.

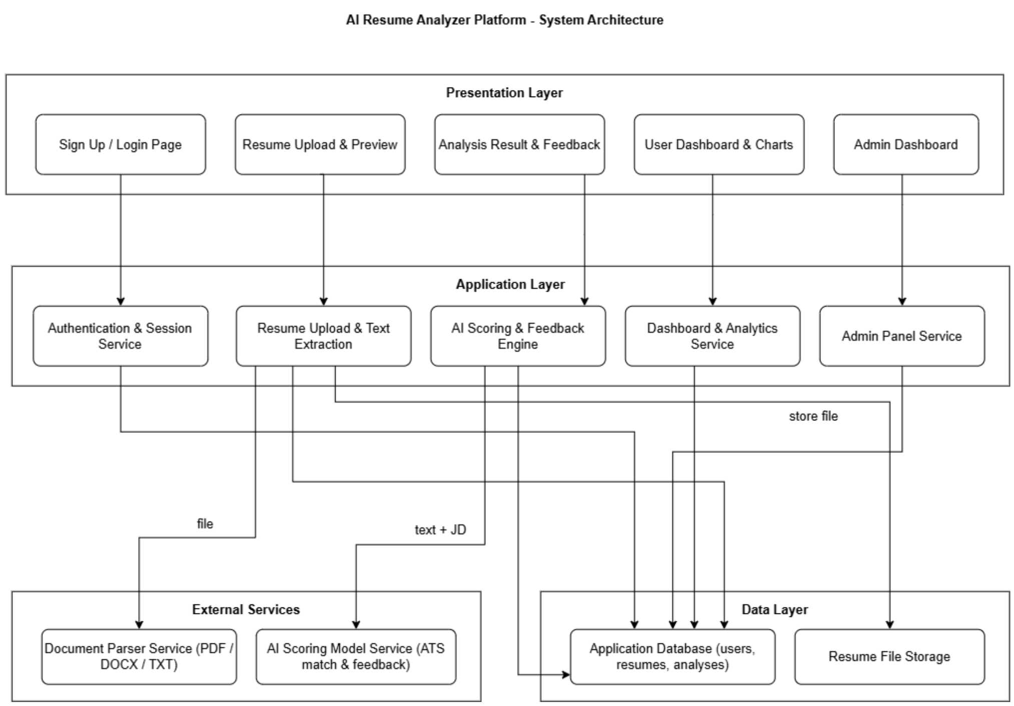

Figure 1: Overall System Architecture of the AI Resume Analyzer Platform

1.2 Objectives

- To give job seekers free and quick feedback on their resume.
- To show how well a resume matches a specific job description, expressed as an ATS score.
- To point out the skills and keywords that are missing from the resume.
- To keep a history of past analyses so that a user can see progress over time.
- To give administrators a safe way to manage user accounts and watch system activity.
- To keep resume data private by performing the analysis on a locally hosted model.

1.3 System Benefits

1.3.1 Immediate Feedback

A user receives a score and written feedback within seconds, instead of waiting for a rejection that never explains itself.

1.3.2 Job Specific Matching

The resume is compared against one particular job description, so the advice is specific to the post the user is applying for rather than generic.

1.3.3 Clear Improvement Path

Feedback is grouped into Skills, Content Quality, Structure and Writing Style, and the missing keywords are listed as separate tags, so the user knows exactly what to change.

1.3.4 Progress Tracking

Every analysis is stored and shown on a Score Over Time chart, so a user can confirm that a rewritten resume actually scores better.

1.3.5 Data Privacy

The AI model runs on the local machine, so a resume, which is a personal document, is never uploaded to a third-party service.

1.3.6 No Running Cost

Because the model is open source and locally hosted, there is no per request billing, which makes the service free for the people who need it most.

1.3.7 Centralised Administration

Administrators see system statistics, a searchable user list and activity logs from one dashboard.

1.3.8 Role Based Access

A single login separates the user experience from the administrator experience, and the permission check is enforced at the API boundary.

1.3.9 Extensible Design

The AI model sits behind a connector, so replacing it or moving it to a stronger server is a configuration change rather than a rewrite.

1.4 Software Process Model

The project followed Scrum, an agile process model, with short sprints of about one week. The work was kept on a Scrum board, divided into six sprints, and each sprint was mapped to one epic of the system.

Each sprint began with sprint planning, where tasks were moved from the product backlog into the sprint backlog. Work then proceeded through design, coding and testing inside the sprint, with a short daily stand-up to keep the three members aligned. At the end of the sprint the working increment was demonstrated, and the retrospective sent lessons and unfinished work back into the product backlog.

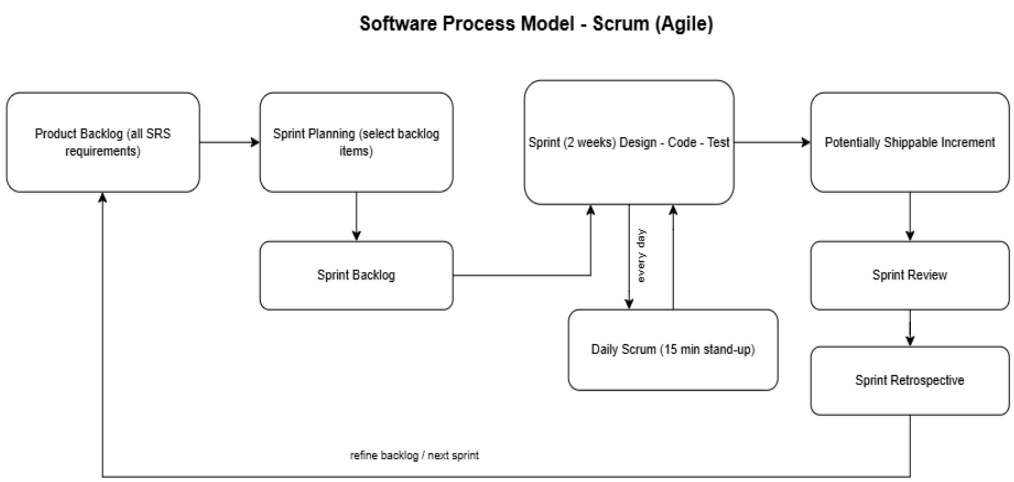

Figure 2: The Scrum process model used in this project

Work was divided by layer so that the three members could work in parallel: Ehetisum Sharif took the C# API and the AI integration, Md. Sajid took the React frontend and state management, and Nishat Tasnim took the database and document processing.

1.5 Why the Scrum Model Was Chosen

Scrum was chosen over the waterfall model for one concrete reason discovered during feasibility testing: the behaviour of the AI model could not be predicted from a document, only from running it. A prototype that sent one resume and one job description to Llama 3 showed that the model produced useful feedback but did not always return clean JSON, and that it slowed down when the machine was busy. A process that allowed the requirements to change after each sprint was therefore safer than a process that fixed everything at the start.

- The requirements around the AI module changed twice after real behaviour was observed, and Scrum absorbed those changes without reopening a signed specification.
- The schedule feasibility check showed that the first backlog did not fit one semester. Scrum made it natural to move job recommendation and mock interview to a future release and keep five modules complete instead of seven half finished.
- A working increment at the end of every sprint meant that integration problems, such as the CORS policy between the API and the client, surfaced early rather than at the end.
- With only three members, the daily stand-up and a single shared backlog were enough coordination, so the process overhead stayed low.

CHAPTER 2: REQUIREMENT ENGINEERING

2.1 User Requirements

The user requirements describe what the two types of user expect from the system, written in the language of the user rather than the language of the developer.

2.1.1 Job Seeker Requirements

- I want to create an account and log in securely.
- I want to upload my resume and see that the correct file was uploaded.
- I want to enter the job description of the post I am applying for.
- I want a score that tells me how well my resume matches that job.
- I want written feedback that tells me what to improve.
- I want to know which skills and keywords are missing from my resume.
- I want to see all my past analyses and whether my score is improving.
- I want to compare my resume and the job description side by side.
- I want to rate the analysis so that the team knows whether it was useful.

2.1.2 Administrator Requirements

- I want a separate dashboard after I log in as an administrator.
- I want to see how many users are registered and how many resumes have been analysed.
- I want to see the average ATS score and the number of active sessions.
- I want to search a paginated list of all users.
- I want to change a user account between Active and Banned.
- I want to review system activity, AI processing status and error logs.

2.2 System Requirements

2.2.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i5 (10th generation) or better
Memory	16 GB RAM (8 GB minimum; the local AI model is memory hungry)
Storage	256 GB SSD with at least 20 GB free
Graphics	Dedicated GPU recommended for faster Llama 3 inference
Network	Broadband internet connection for development and package downloads
Peripherals	Standard keyboard, mouse and display

Table 1: Hardware requirements

2.2.2 Software Requirements

Component	Technology Used
Operating System	Windows 10 or Windows 11
Backend Framework	ASP.NET Core 8 Web API (C#)
Authentication	ASP.NET Core Identity with JWT tokens
Data Access	Entity Framework Core 8
Database	Microsoft SQL Server
Frontend Framework	React 19 built with Vite
Styling	Tailwind CSS
State Management	Zustand
HTTP Client	Axios
AI Model	Llama 3 (Meta) hosted locally through Ollama at http://localhost:11434
AI Connector	Microsoft Semantic Kernel
Document Processing	iTextSharp for PDF text extraction, Magick.NET for preview rendering at 150 DPI
API Testing	Swagger
IDE and Tools	Visual Studio 2022, Visual Studio Code, draw.io, Git and GitHub

Table 2: Software requirements

2.3 Functional Requirements

The functional requirements state what the system must do. Each requirement carries an identifier that is used again in the test cases in Chapter 8.

ID	Functional Requirement
FR-01	The system shall let a new user create an account using name, email address, password and confirm password.
FR-02	The system shall check that the password and the confirm password match and shall show "Passwords do not match" without reloading the page.
FR-03	The system shall let a registered user or an administrator log in with an email address and a password.
FR-04	The system shall show "Invalid email or password" when the credentials do not match a stored record.
FR-05	The system shall create a secure session carrying the user role and redirect an administrator to the Admin Dashboard and a user to the Resume Analysis Dashboard.
FR-06	The system shall end the current session and return to the login page when the user clicks Logout.
FR-07	The system shall let a user upload a resume by drag and drop or by selecting a file.
FR-08	The system shall accept only the permitted document formats up to a maximum size of 5 MB and shall show an error immediately for any other file.
FR-09	The system shall display a preview of the uploaded resume without reloading the page.
FR-10	The system shall extract the plain text of the uploaded resume using a document parsing tool and store it.
FR-11	The system shall let the user enter the target job description together with the job title and company name.
FR-12	The system shall send the extracted resume text and the job description to the AI model when the user clicks Analyze Resume.
FR-13	The system shall display an "Analyzing Resume..." message while the analysis is running.
FR-14	The system shall generate an ATS match score between 0% and 100%.
FR-15	The system shall group the AI feedback into four categories: Skills, Content Quality, Structure and Writing Style.
FR-16	The system shall identify the important skills and keywords that are missing from the resume.
FR-17	The system shall display the missing keywords as separate recommendation tags.
FR-18	The system shall allow a rating and a comment to be submitted only after an analysis has completed successfully.
FR-19	The system shall show an honest error message instead of a score when an analysis fails.
FR-20	The system shall show all previously uploaded resumes with their ATS scores and analysis dates on the user dashboard.

FR-21	The system shall display a Score Over Time chart built from the past analyses of the user.
FR-22	The system shall let the user select a past analysis and view the resume and the job description side by side.
FR-23	The system shall provide a separate Admin Dashboard for administrators after login.
FR-24	The system shall display total registered users, total resumes analysed, average ATS score and active user sessions on the Admin Dashboard.
FR-25	The system shall provide a searchable and paginated list of all users with their profile information and role.
FR-26	The system shall let an administrator change a user account status between Active and Banned.
FR-27	The system shall let an administrator view system activity logs, AI processing status and database error logs.

Table 3: Functional requirements

2.4 Non-Functional Requirements

The non-functional requirements state the qualities the system must have while it performs those functions.

NFR-1: Performance

- Resume upload and file validation shall complete quickly.
- Resume preview and text extraction shall finish within 5 seconds.
- AI analysis shall normally finish within 15 seconds.
- Dashboard information shall load without noticeable delay.
- The system shall support multiple users at the same time without a significant loss of performance.

NFR-2: Security

- Only authorised administrators shall access administrative features.
- User passwords shall be securely hashed before being stored.
- All protected pages and API endpoints shall require secure authentication.
- Unauthorised users shall not be able to reach admin pages or sensitive data.

NFR-3: Reliability

- Resume text shall be extracted accurately and consistently.
- File processing shall not lose or corrupt resume data.
- Analysis results shall remain correct even when unexpected errors occur.

NFR-4: Usability

- The application shall use a clean and consistent dark theme.
- Navigation shall be simple for both users and administrators.
- Error messages, warnings and success notifications shall be clear.

NFR-5: Maintainability

- The system shall follow a layered architecture that keeps the parts of the application independent.
- New features such as job recommendation or mock interview shall be easy to add later.
- The source code shall be well organised and documented.

NFR-6: Scalability

- The system shall handle growth in registered users, uploaded resumes and analysis history.
- The system shall handle growth in simultaneous AI analysis requests.

NFR-7: Availability

- The system shall be available so that users can view previous analyses, upload new resumes and receive AI analysis whenever needed.

NFR-8: Data Integrity

- The system shall maintain accurate relationships between user accounts, uploaded resumes and AI analysis results.
- Stored information shall remain complete, consistent and free from corruption.

2.5 Security Requirements

Security was treated as a design constraint rather than a feature added at the end, because the frontend is a separate application and therefore cannot be trusted to enforce any rule.

Area	Measure Taken
Authentication	Accounts are created and verified through ASP.NET Core Identity. A JSON Web Token is issued at login and carries the identity and the role of the user.
Authorization	Every protected controller carries an authorisation attribute. The permission check is placed at the controller layer because the frontend is separate and hiding a button in the client protects nothing.
Session Management	The API is stateless. Each request carries its own token, so no session is held in server memory and any instance can answer a request.
Password Protection	Passwords are never stored in plain text. Only the hash produced by Identity is written to the database.

Input Validation	File type and file size are validated on upload, and sign up and login input is validated before it reaches the database.
Database Security	All database access goes through Entity Framework Core, which parameterises queries and removes direct SQL string building.
Unauthorized Access Prevention	Middleware order is fixed as CORS, then authentication, then authorization, so the server always knows who the caller is before it checks a permission.
Data Privacy	The AI model runs locally through Ollama, so no resume is ever sent to a third-party service.

Table 4: Security requirements

2.6 Use Case Diagram

The use case diagram shows the interaction between the system and its actors. The two primary actors are the Job Seeker and the Administrator. The Document Parser Service and the AI Scoring Model Service are secondary actors, because the system depends on them to complete certain use cases but neither of them initiates any action.

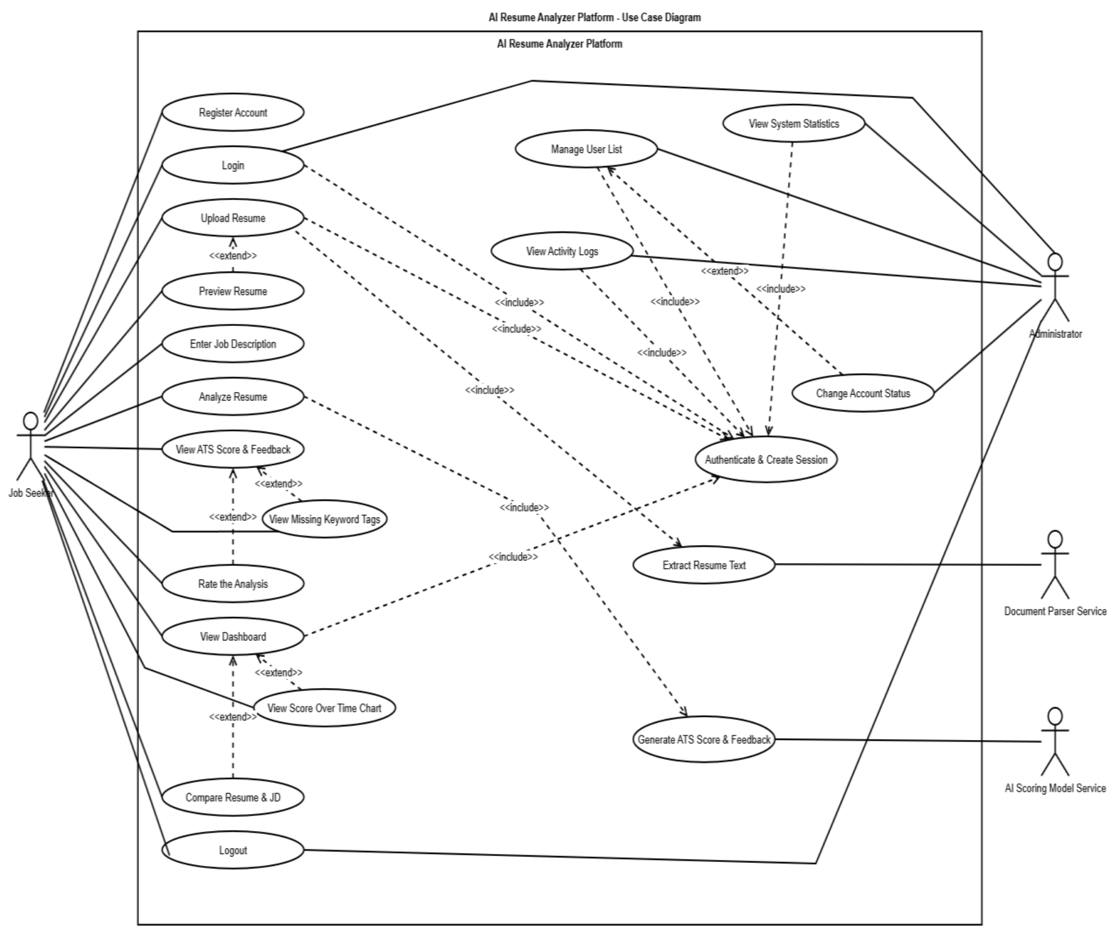

Figure 3: Use Case Diagram of the AI Resume Analyzer Platform

2.7 Use Case Text

Each use case shown above is described below in text form. Following the standard format, every use case records its title, the actor who takes part in it, and a description written in the first person from the point of view of the actor.

2.7.1 Register Account

Use-case title:	Register Account
Actor:	Job Seeker
Description:	I open the application and choose Sign Up. I enter my name, my email address, a password and the confirm password. If the two passwords do not match, the system shows me a “Passwords do not match” message straight away without reloading the page. If the email address is already registered, the system tells me that as well. When everything is correct the system stores my password in hashed form, creates my account and shows me a registration successful message so that I can log in.

2.7.2 Login

Use-case title:	Login
Actor:	Job Seeker, Administrator
Description:	I enter my email address and my password on the login page and click Login. The system checks my details against the database. If the email or the password is wrong I get an “Invalid email or password” message and I can try again. If my details are correct the system creates a secure session that also holds my role, and then sends me to the Admin Dashboard if I am an administrator or to the Resume Analysis Dashboard if I am a job seeker.

2.7.3 Logout

Use-case title:	Logout
Actor:	Job Seeker, Administrator
Description:	I click the Logout button. The system ends my current session and returns me to the login page, so that nobody else can use my account from the same browser.

2.7.4 Upload Resume

Use-case title:	Upload Resume
Actor:	Job Seeker
Description:	I drag and drop my resume into the upload area, or I select the file from my computer. The system accepts only PDF, DOCX and TXT files up to 5 MB. If

	my file does not meet these conditions I immediately get an error message and I can upload a different file.

2.7.5 Preview Resume

Use-case title:	Preview Resume
Actor:	Job Seeker
Description:	After my file is accepted, I see a preview of my resume on the same page without the page reloading, so I can confirm that I uploaded the correct document.

2.7.6 Extract Resume Text

Use-case title:	Extract Resume Text
Actor:	Job Seeker, Document Parser Service
Description:	Once my resume is accepted, the system sends the file to the document parser, which reads the file and returns the plain text. The system stores that extracted text so it is ready for analysis.

2.7.7 Enter Job Description

Use-case title:	Enter Job Description
Actor:	Job Seeker
Description:	I paste or type the job description of the position I am applying for, together with the job title and the company name, so that my resume can be compared against that specific job.

2.7.8 Analyze Resume

Use-case title:	Analyze Resume
Actor:	Job Seeker
Description:	I click Analyze Resume. The system sends my extracted resume text and the job description to the AI model and shows me an “Analyzing Resume...” message while it waits. If the analysis fails I am shown an error and I can try again.

2.7.9 View ATS Score and Feedback

Use-case title:	View ATS Score and Feedback
Actor:	Job Seeker

Description:	When the analysis finishes I see my ATS match score between 0% and 100%, and the feedback grouped into four categories: Skills, Content Quality, Structure and Writing Style.

2.7.10 View Missing Keyword Tags

Use-case title:	View Missing Keyword Tags
Actor:	Job Seeker
Description:	Along with my score I see the important skills and keywords that are missing from my resume, shown as separate recommendation tags, so I know exactly what to add to improve my resume.

2.7.11 Rate the Analysis

Use-case title:	Rate the Analysis
Actor:	Job Seeker
Description:	After an analysis has finished successfully, I can give the analysis a rating and write a short comment. I cannot submit a rating before the analysis has completed.

2.7.12 View Dashboard

Use-case title:	View Dashboard
Actor:	Job Seeker
Description:	I open my dashboard and see every resume I have uploaded, the ATS score each one received and the date of each analysis. If I have not run any analysis yet, the system tells me so.

2.7.13 View Score Over Time Chart

Use-case title:	View Score Over Time Chart
Actor:	Job Seeker
Description:	On my dashboard I see a Score Over Time chart built from my past analyses, so I can tell whether my resume is improving across attempts.

2.7.14 Compare Resume and Job Description

Use-case title:	Compare Resume and Job Description
Actor:	Job Seeker
Description:	I select one of my previous analyses and the system shows me the resume and

	the job description side by side, so I can see exactly where the two do not match.

2.7.15 View System Statistics

Use-case title:	View System Statistics
Actor:	Administrator
Description:	After I log in as an administrator I land on a separate Admin Dashboard, where I see the total number of registered users, the total number of resumes analysed, the average ATS score and the number of active user sessions.

2.7.16 Manage User List

Use-case title:	Manage User List
Actor:	Administrator
Description:	I open the user list and search through it. The list is paginated and shows me each user with their profile information and their role.

2.7.17 Change Account Status

Use-case title:	Change Account Status
Actor:	Administrator
Description:	From the user list I select a user and change their account status between Active and Banned. The system saves the change and shows me the refreshed list.

2.7.18 View Activity Logs

Use-case title:	View Activity Logs
Actor:	Administrator
Description:	I open the activity logs and review the system activity, the AI processing status and the database error logs, so that I can monitor how the platform is behaving.

CHAPTER 3: ANALYSIS MODELLING

3.1 Introduction to Analysis Modelling

The analysis model describes what the customer requires and provides the basis for the design that follows. It is built from scenario based elements, which are the use case diagram, the use case text, the activity diagrams and the swim lane diagrams; class based elements, which are the CRC model and the class diagram; behavioural elements, which are the sequence diagrams; and flow oriented elements, which are the data flow diagrams presented later in Chapter 7.

3.2 Activity Diagrams

An activity diagram shows the flow of activities inside one module, including the decisions that change the path and the points where a flow splits or joins. Guard conditions are written in square brackets, a filled circle marks the initial activity and a bull’s-eye marks the final activity.

3.2.1 Authentication and Session Management

The diagram covers both registration and login in one flow. The decision "Have an account?" splits the two paths, and every validation failure loops back to the input activity instead of ending the flow. After a session is created, the decision "Role = Admin?" selects the dashboard the user is sent to, and both paths join before logout.

Figure 4: Authentication and Session Management Activity Diagram

3.2.2 Resume Upload and Text Extraction

File type and size are checked before anything else happens, so an invalid file never reaches the parser. A second decision handles the case where the parser returns no text at all, which is what a scanned PDF produces.

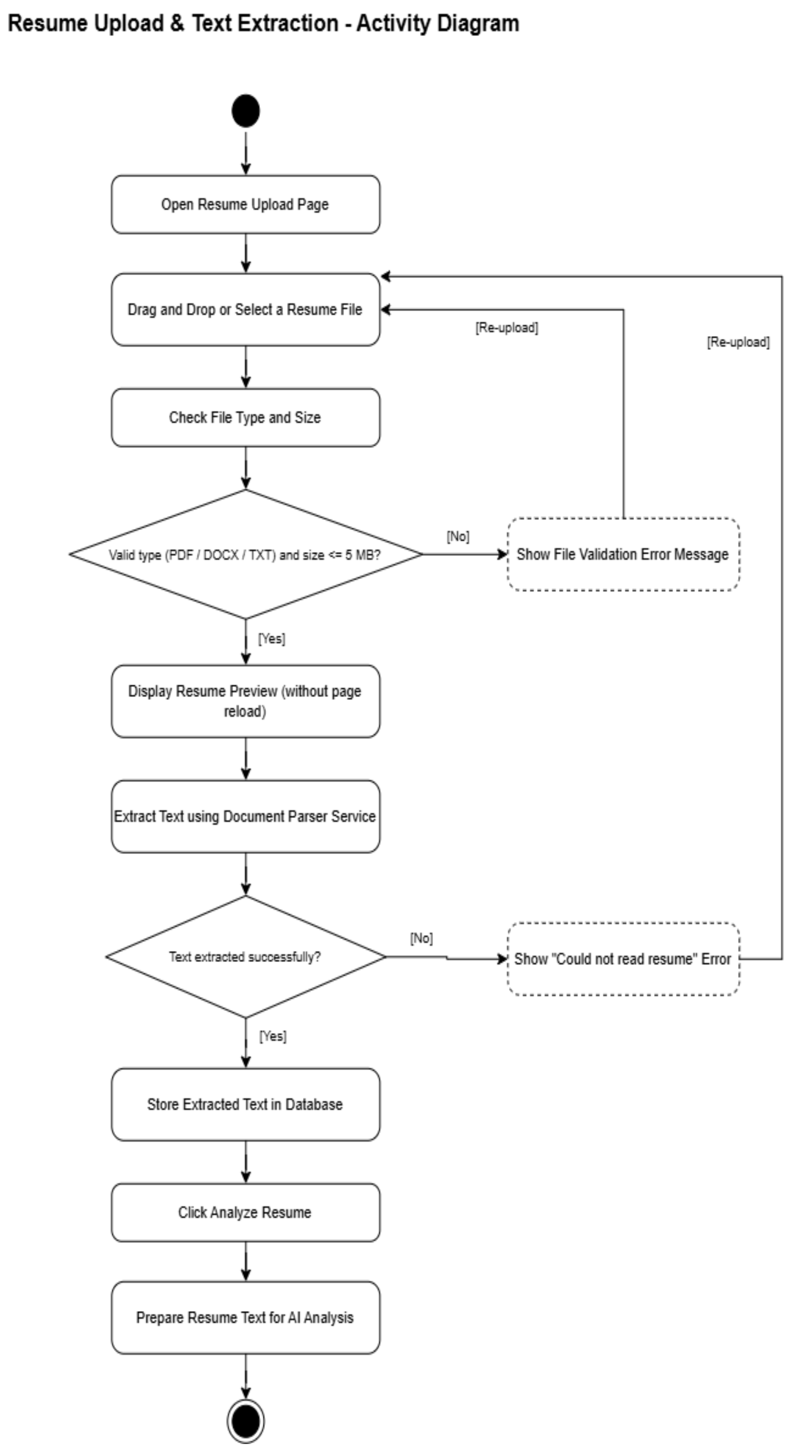

Figure 5: Resume Upload and Text Extraction Activity Diagram

3.2.3 AI Scoring and Feedback Engine

The decision "Analysis completed successfully?" is the guard that stops a failed analysis from reaching the result screen. It is the diagram equivalent of the JSON check in the scoring service, and it is also why a rating cannot be submitted for a failed analysis.

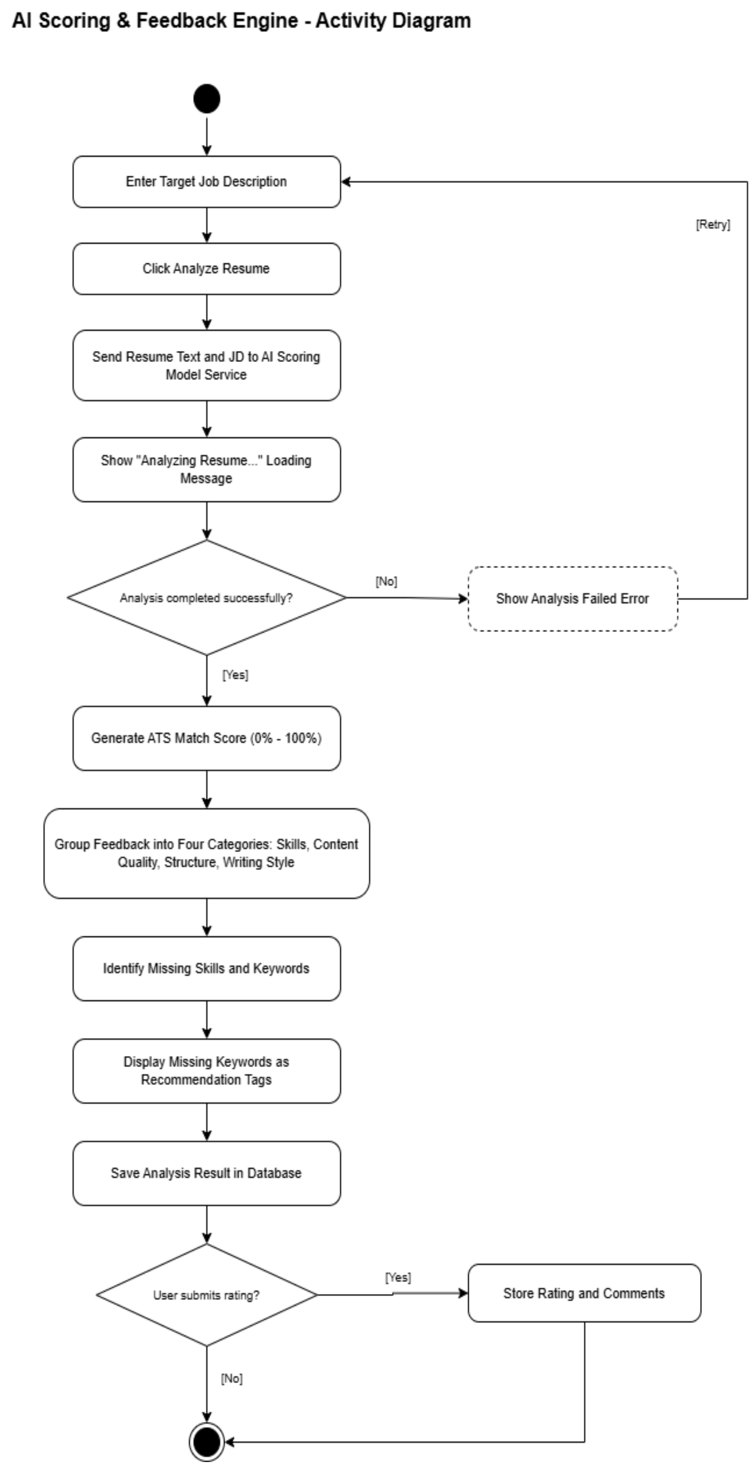

Figure 6: AI Scoring and Feedback Engine Activity Diagram

3.2.4 User Dashboard and Analytics

The flow first checks whether any past analysis exists, because a new user has an empty dashboard. Only then are the list and the chart built, and selecting a record opens the side by side comparison.

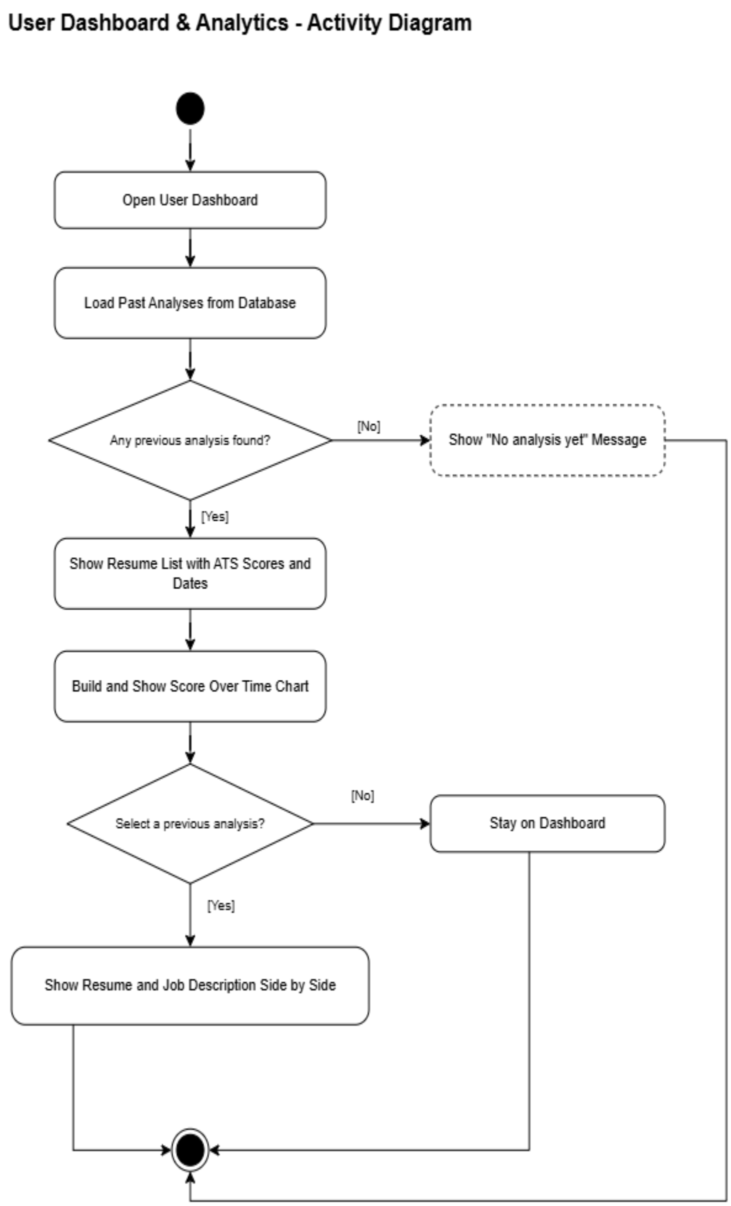

Figure 7: User Dashboard and Analytics Activity Diagram

3.2.5 Admin Panel Management

A single decision selects between the three administrative actions, and all three paths join before the flow ends.

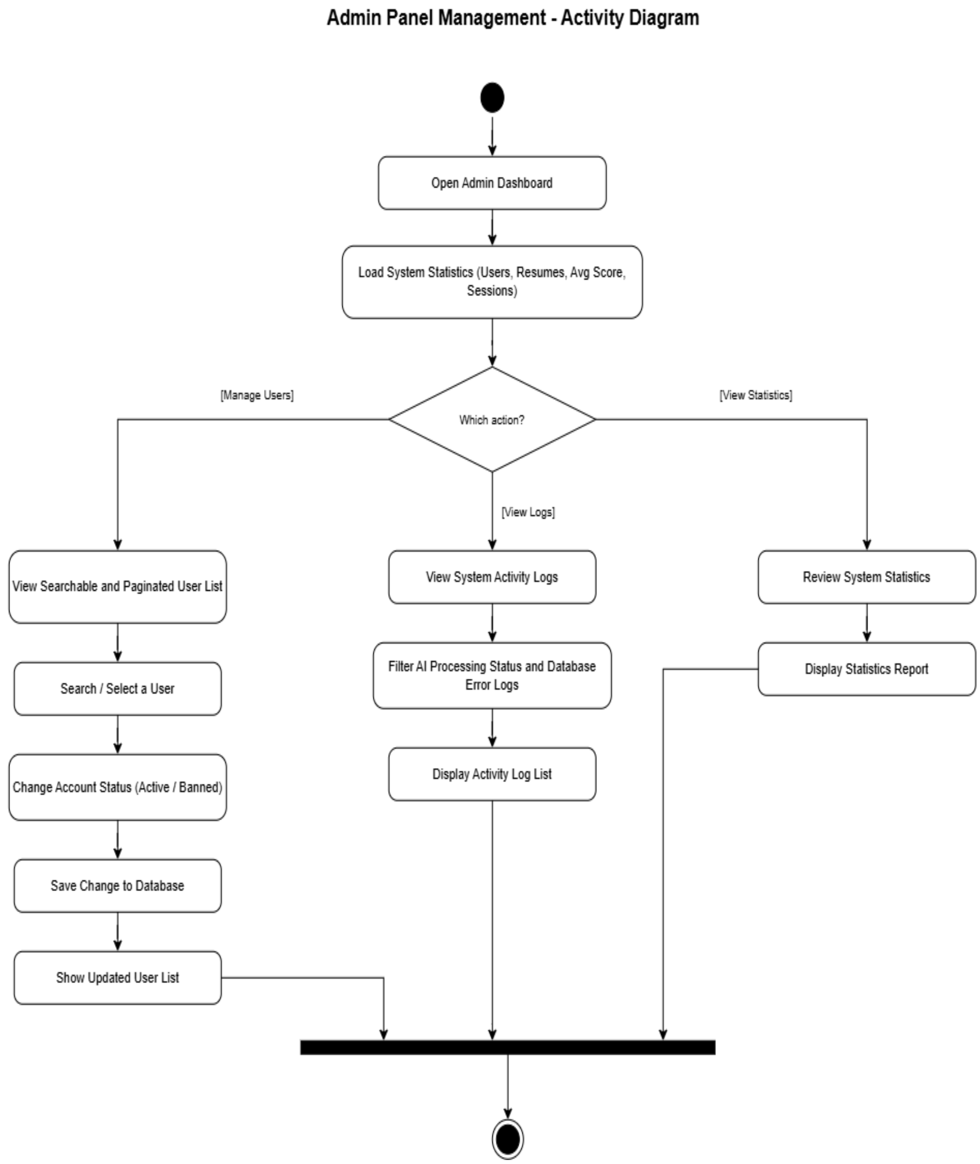

Figure 8: Admin Panel Management Activity Diagram

3.3 Swim Lane Diagrams

A swim lane diagram shows the same flow of activities but also indicates which actor or which part of the system is responsible for each activity. It is a cross-functional flowchart that provides richer information on who does what.

3.3.1 User Registration (Sign Up)

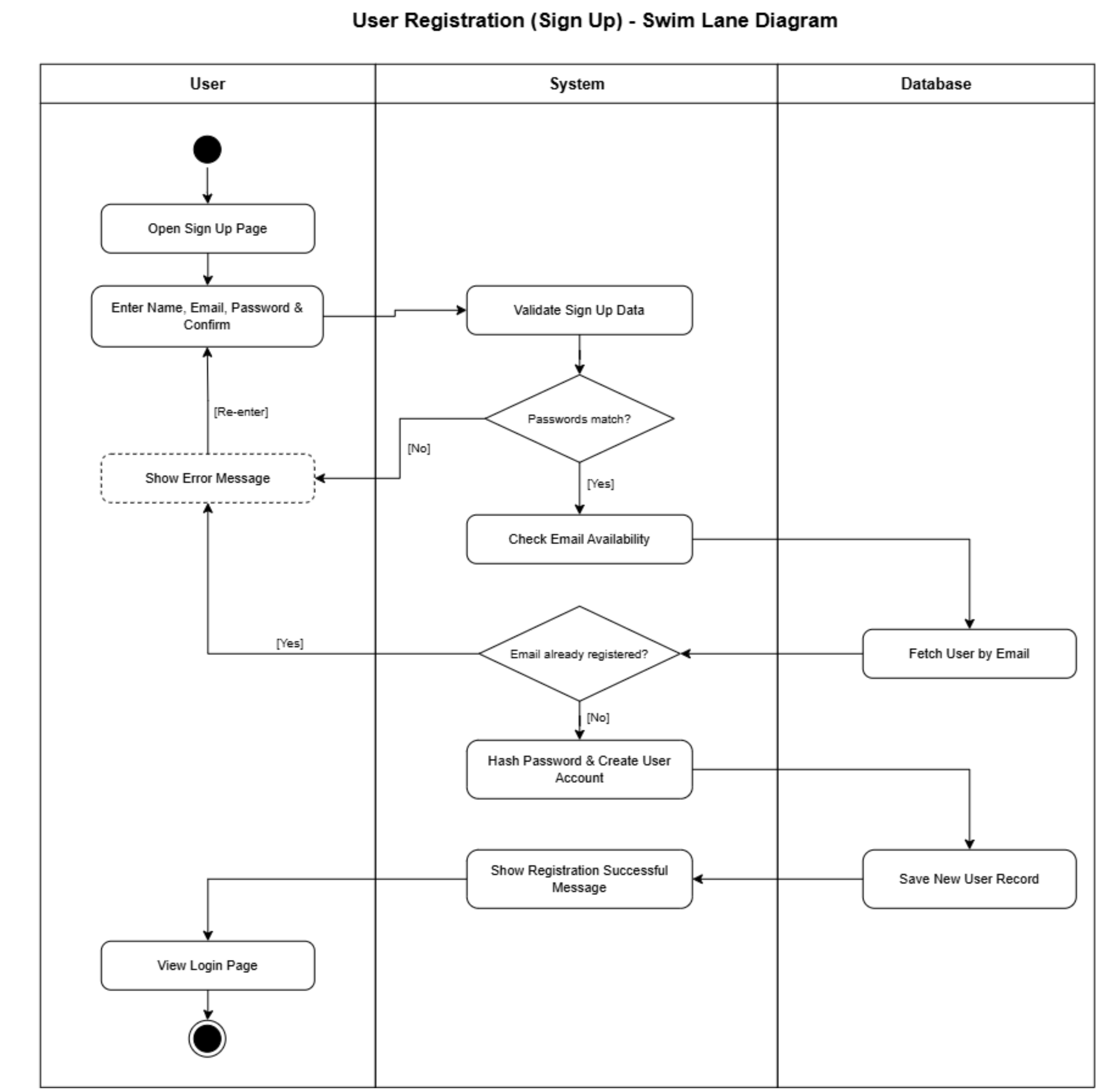

Figure 9: User Registration (Sign Up) Swim Lane Diagram

3.3.2 User Log In and Session Management

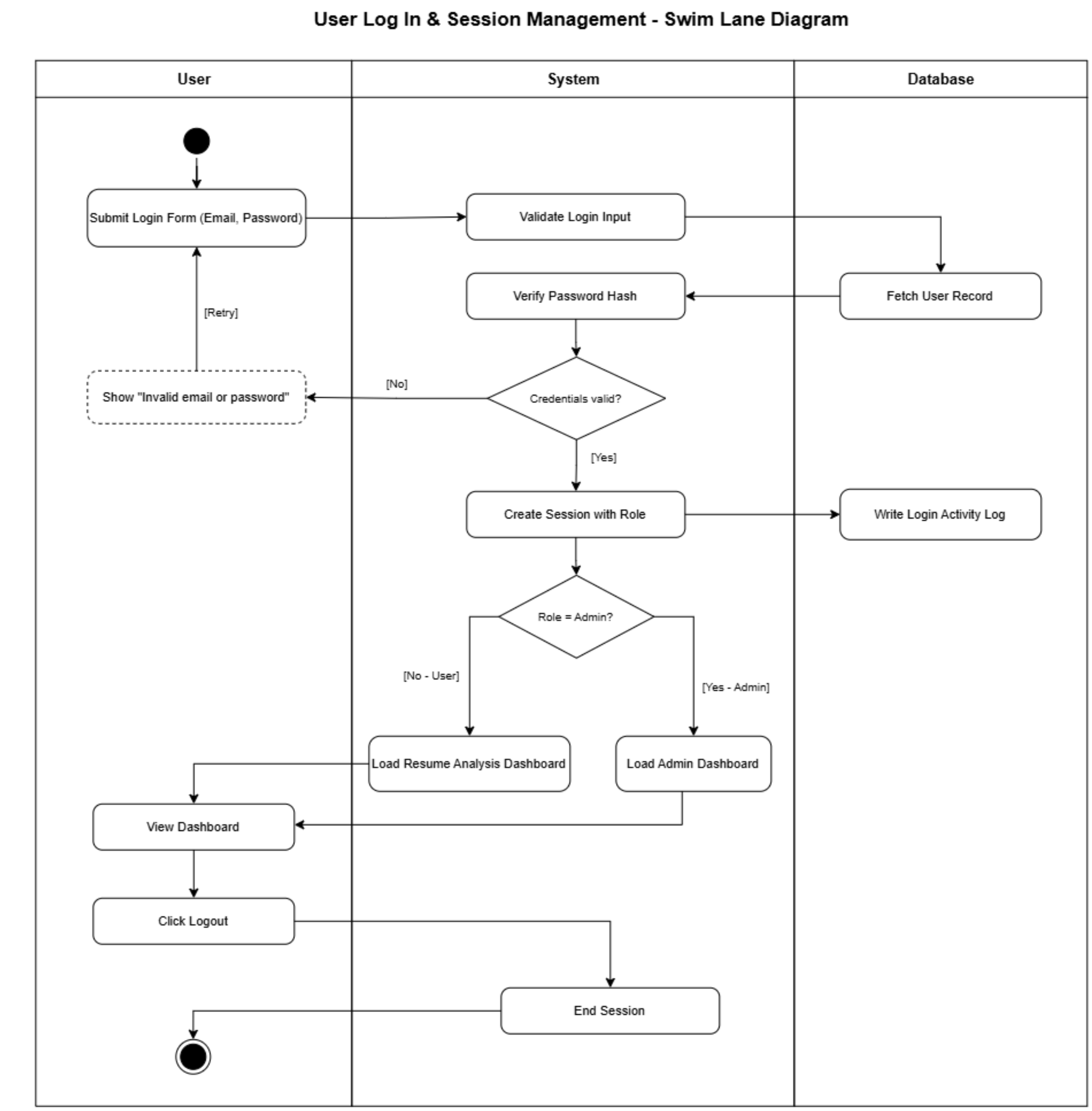

Figure 10: User Log In and Session Management Swim Lane Diagram

3.3.3 Resume Upload and Text Extraction

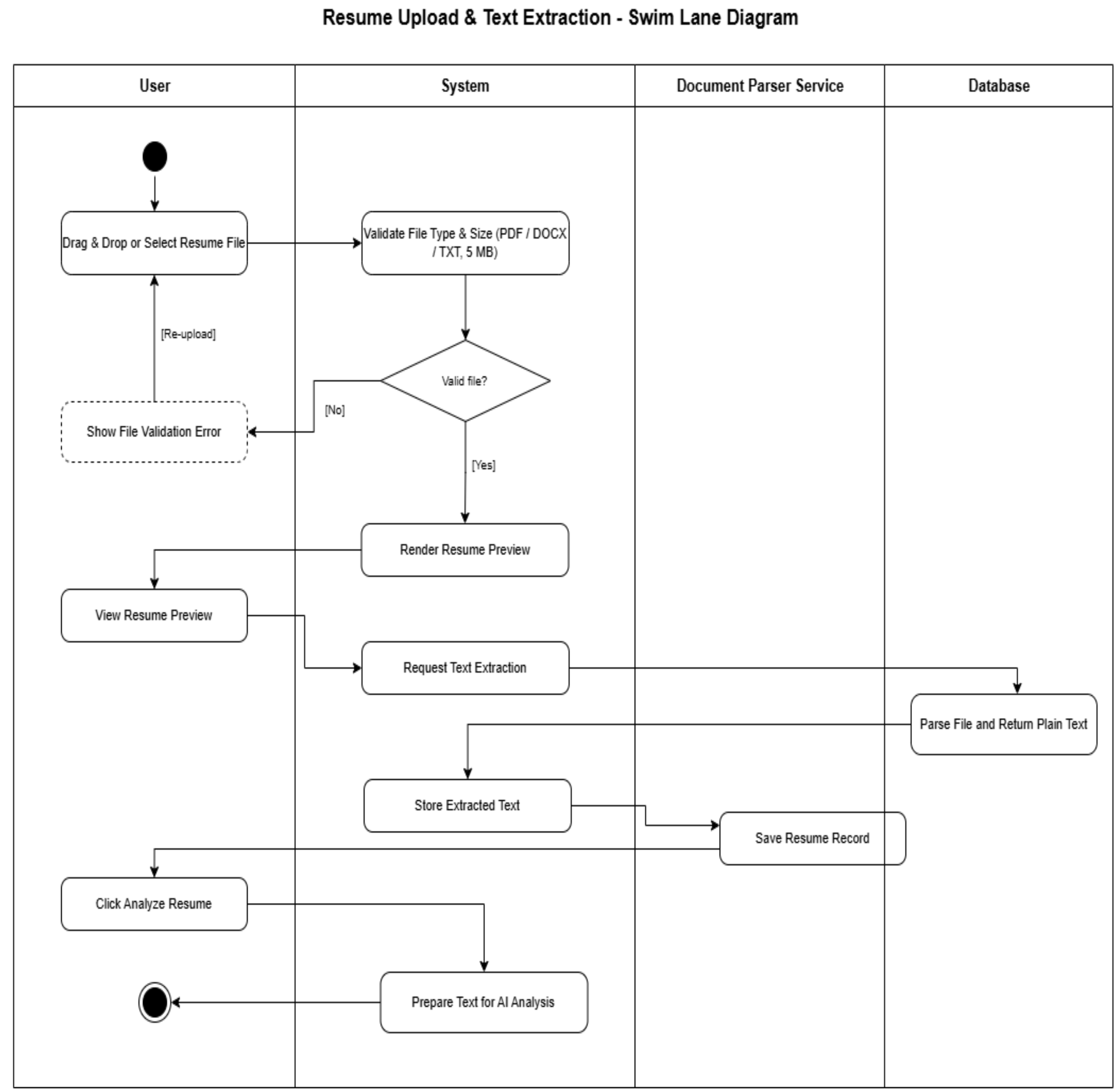

Figure 11: Resume Upload and Text Extraction Swim Lane Diagram

3.3.4 AI Scoring and Feedback

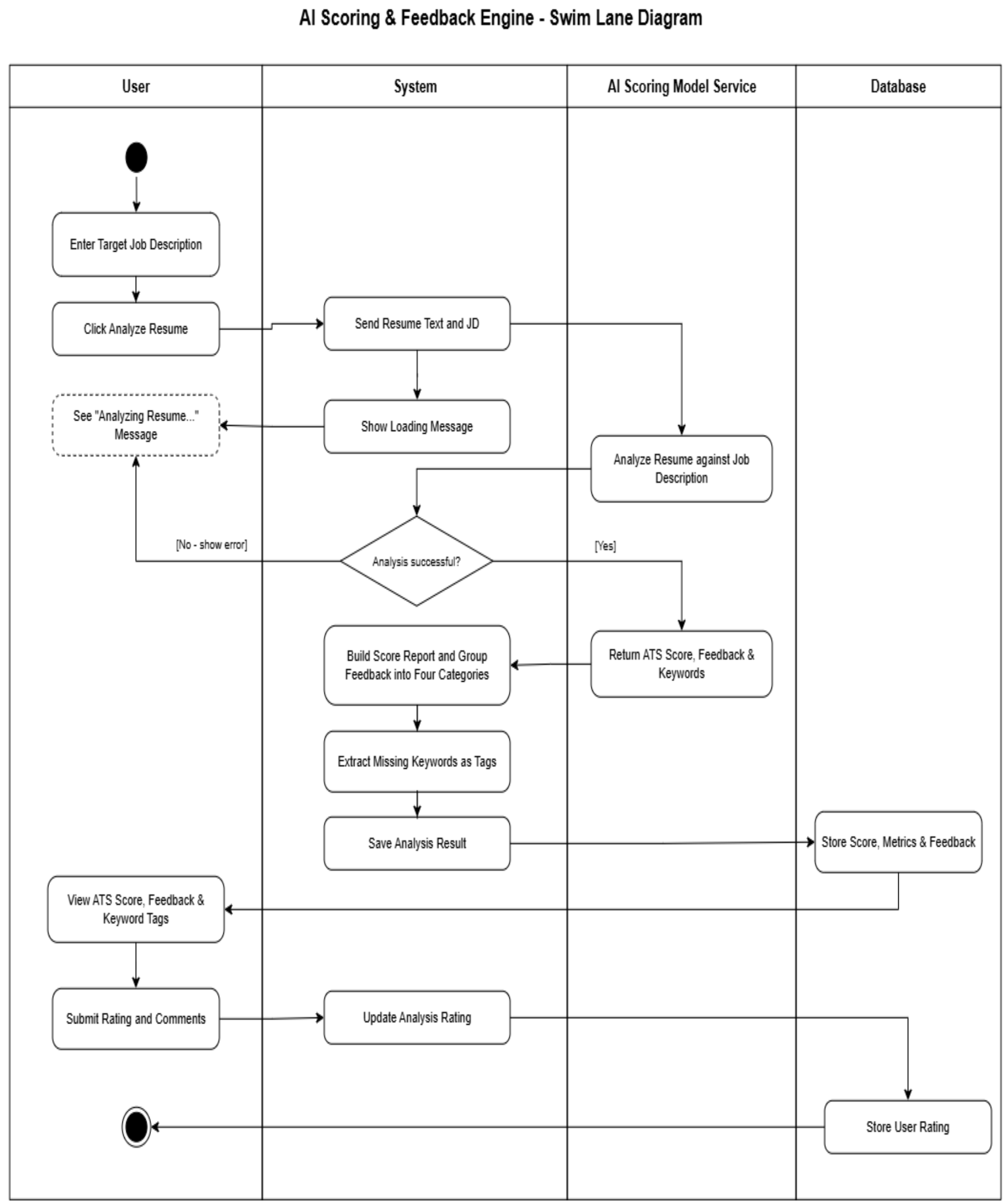

Figure 12: AI Scoring and Feedback Swim Lane Diagram

3.3.5 User Dashboard and Analytics

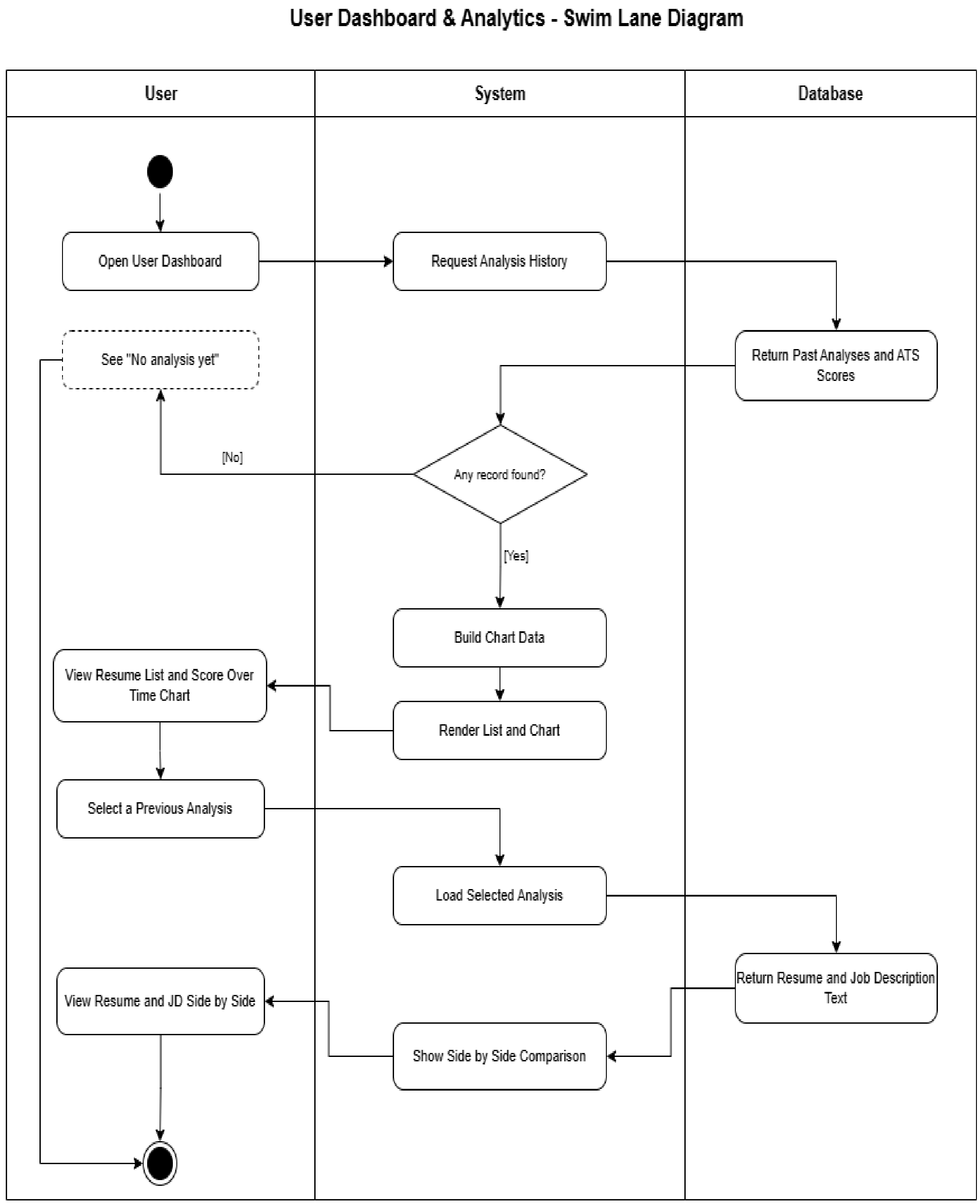

Figure 13: User Dashboard and Analytics Swim Lane Diagram

3.3.6 Admin Panel Management

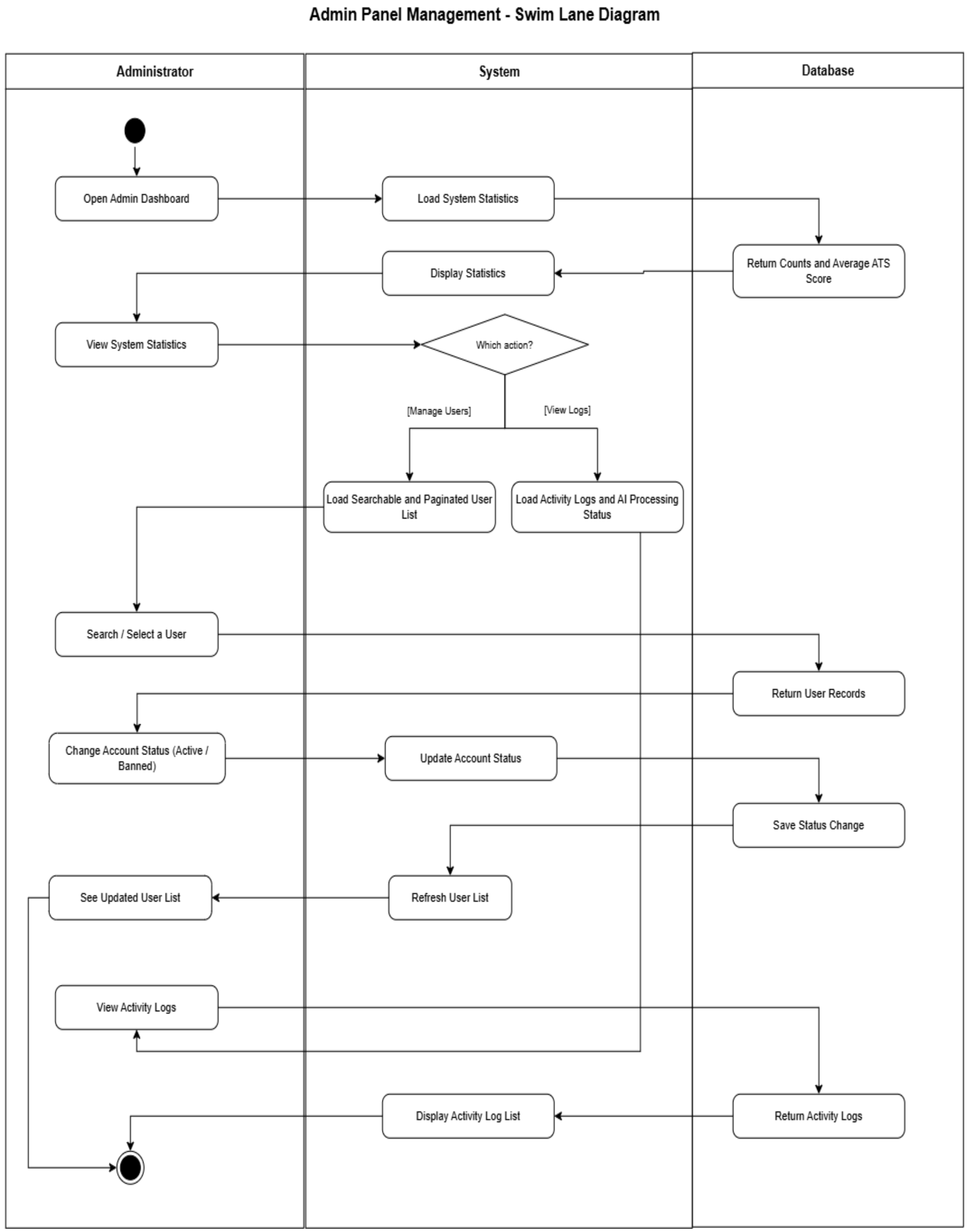

Figure 14: Admin Panel Management Swim Lane Diagram

3.4 Sequence Diagrams

A sequence diagram shows the interaction between objects in chronological order. Each diagram is drawn inside a frame labelled sd, participants are named in the object and class form, and a dashed arrow marks a return message. Combined fragments labelled alt and opt show the alternative and optional paths.

3.4.1 User Registration (Sign Up)

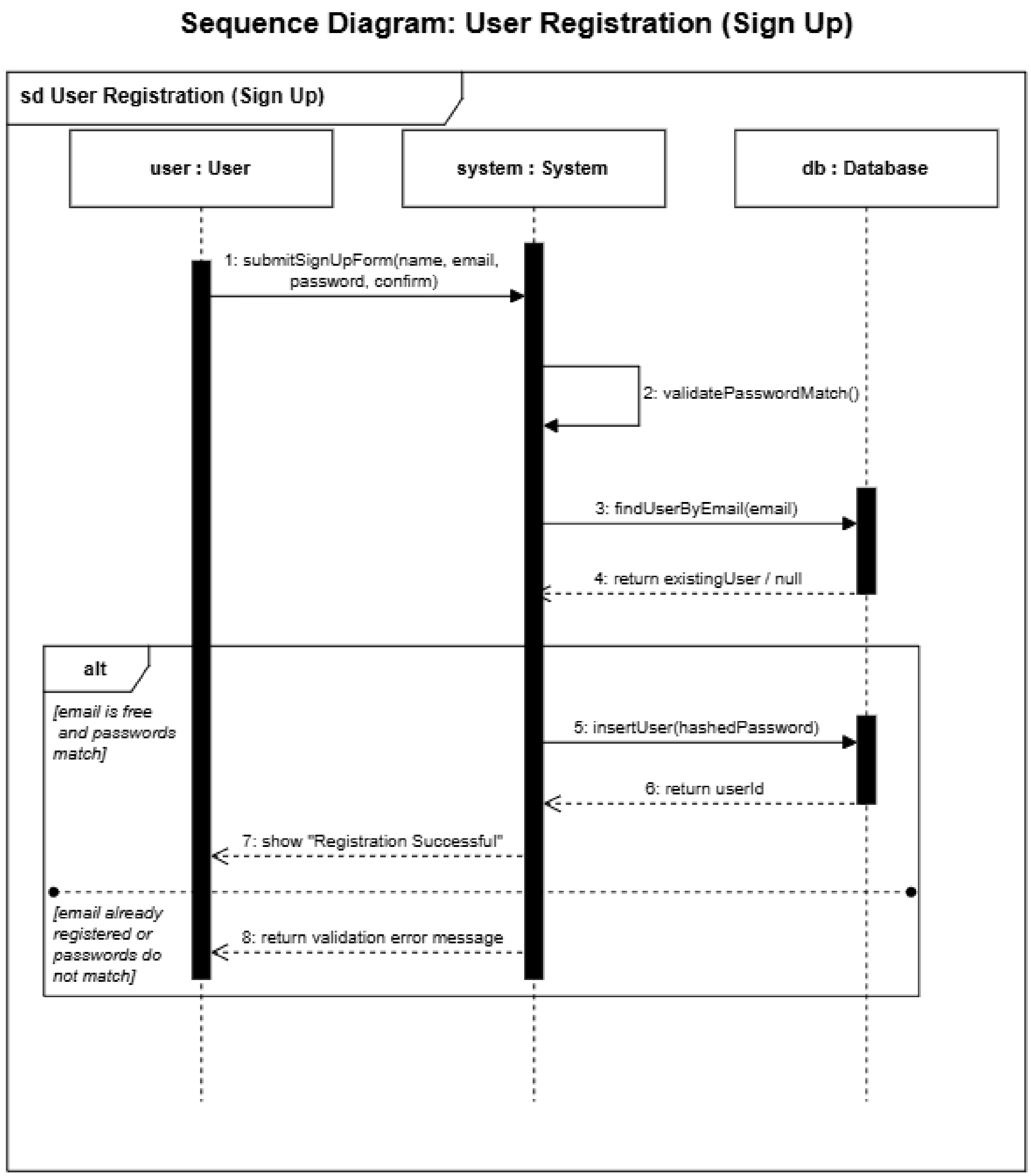

Figure 15: User Registration (Sign Up) Sequence Diagram

3.4.2 Log In and Session Management

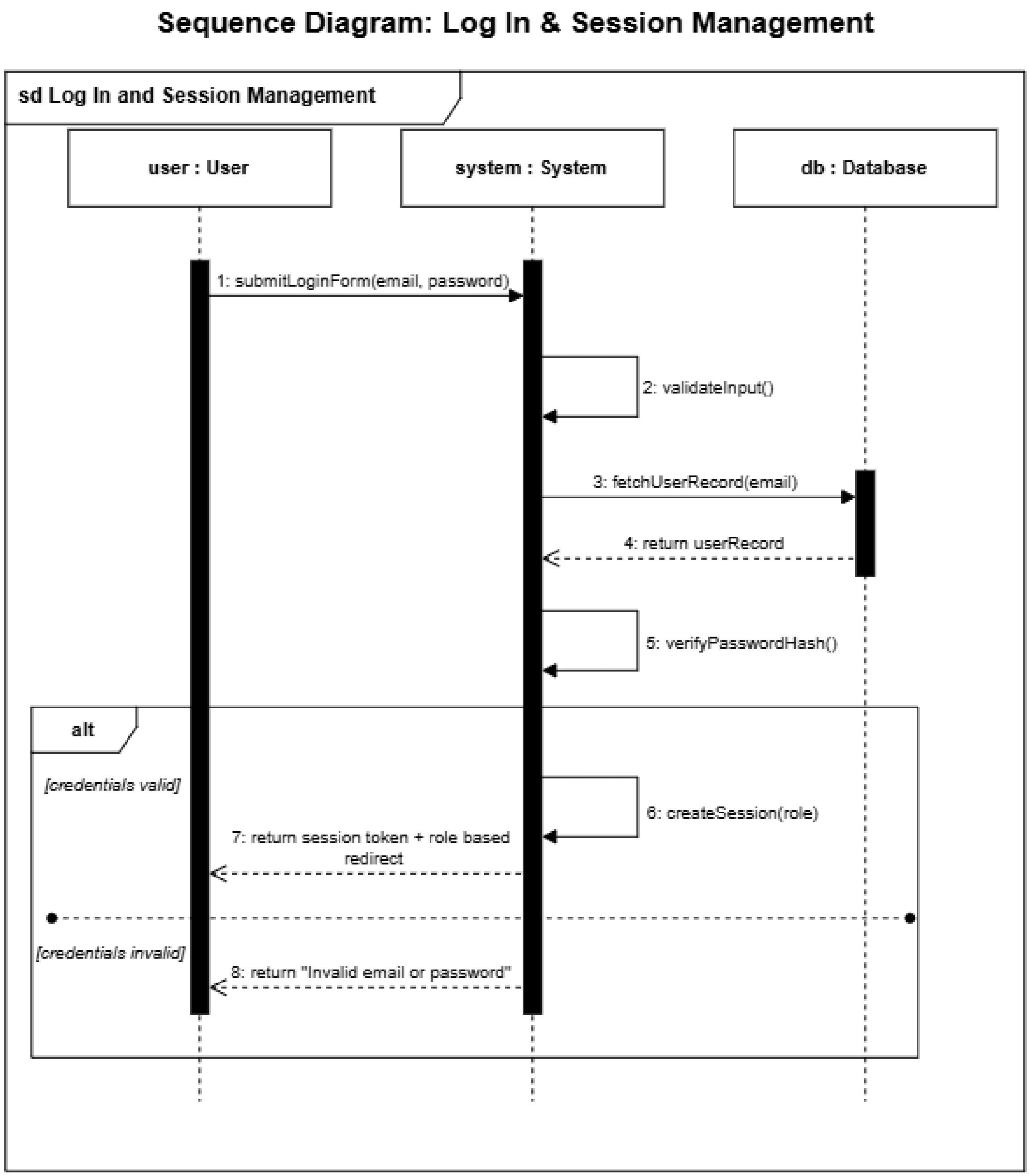

Figure 16: Log In and Session Management Sequence Diagram

3.4.3 Resume Upload and Text Extraction

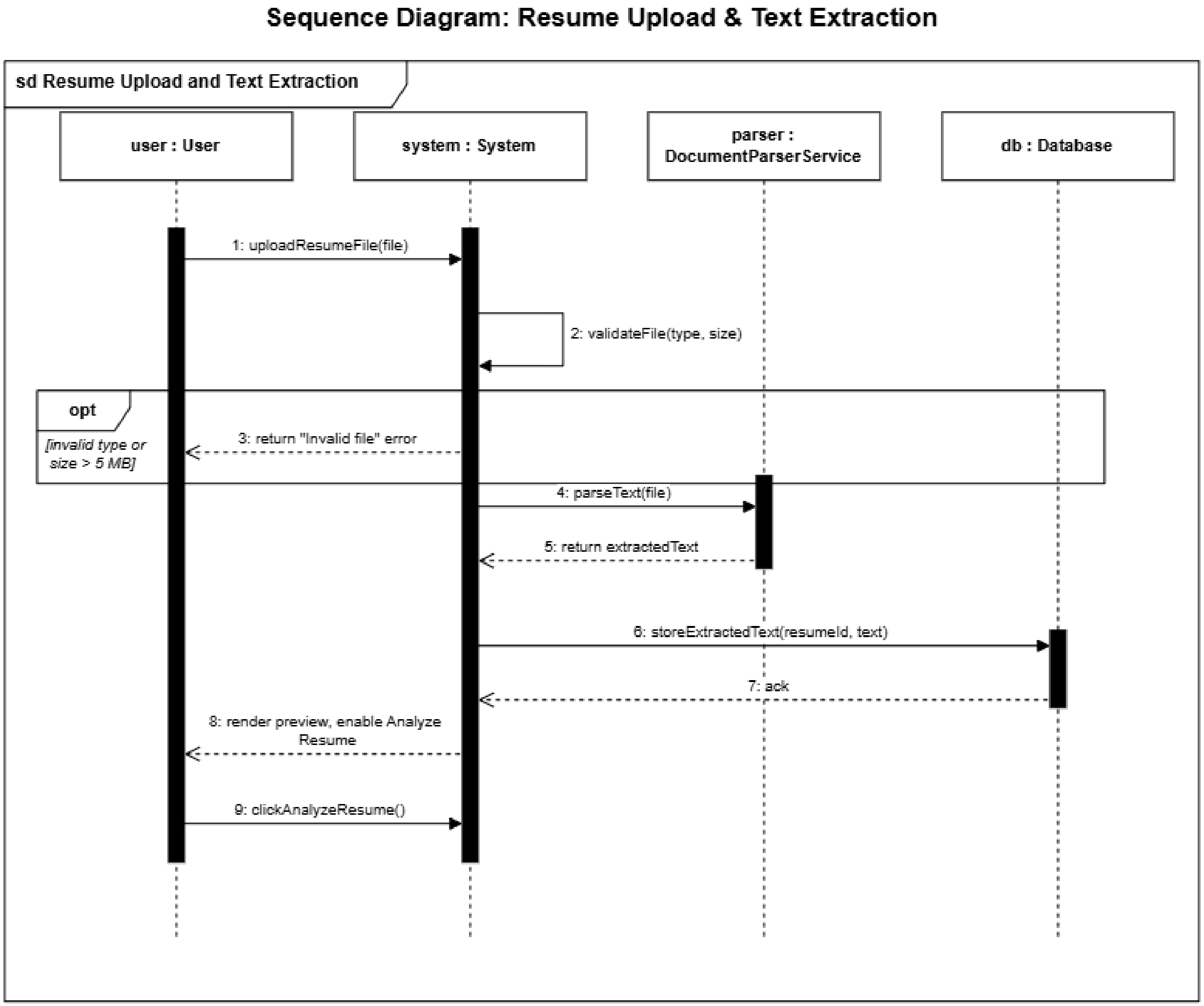

Figure 17: Resume Upload and Text Extraction Sequence Diagram

3.4.4 AI Scoring and Feedback

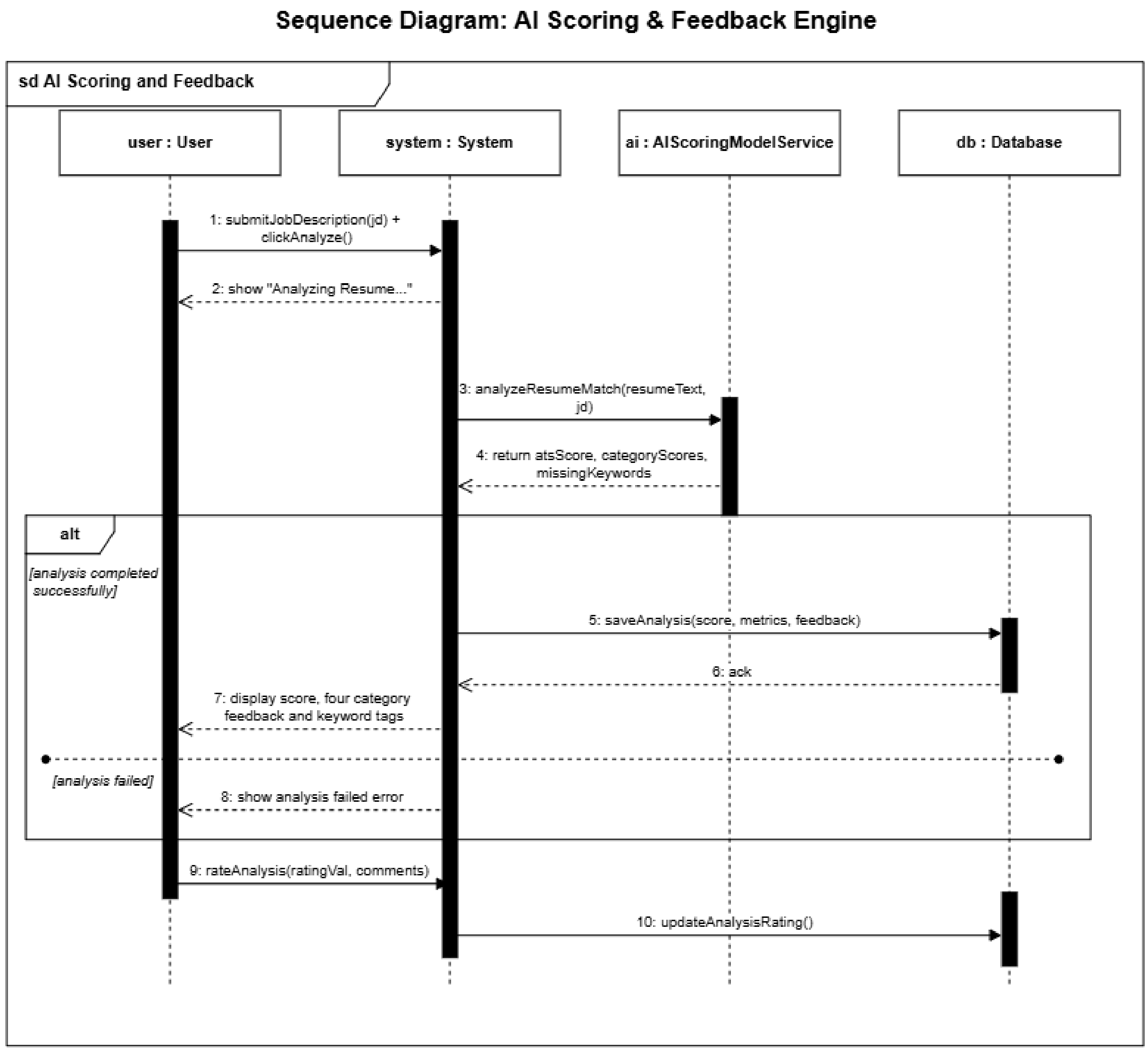

Figure 18: AI Scoring and Feedback Sequence Diagram

3.4.5 User Dashboard and Analytics

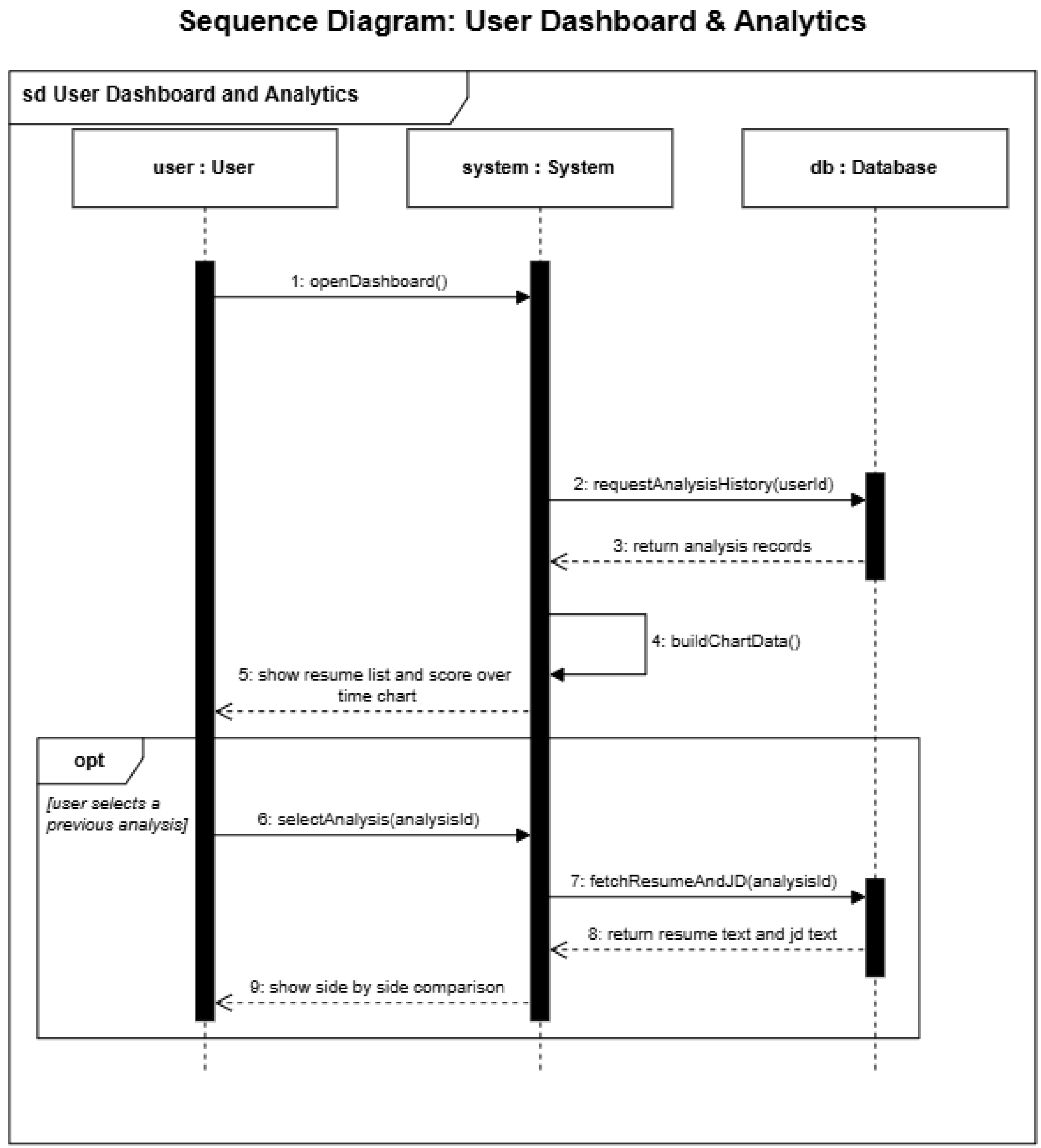

Figure 19: User Dashboard and Analytics Sequence Diagram

3.4.6 Admin Panel Management

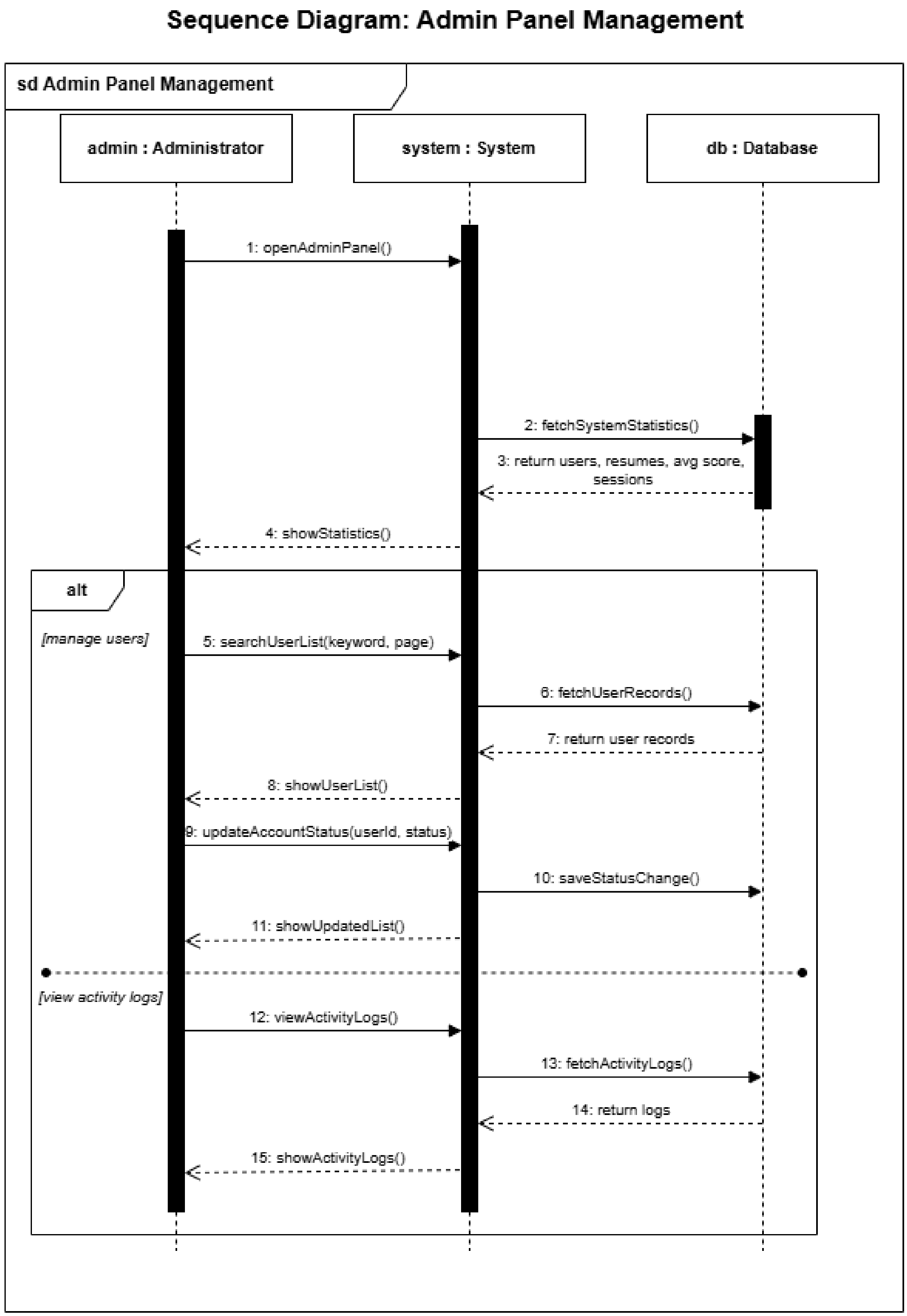

Figure 20: Admin Panel Management Sequence Diagram

3.5 CRC Model

CRC stands for Class, Responsibility and Collaborator. The CRC model identifies the major classes of the system, defines the responsibilities carried by each class, and records the other classes each class must work with in order to complete those responsibilities. Each class is recorded on a CRC index card holding the class name, the class type, the class characteristics, the responsibilities and the collaborators.

The class type is chosen from the standard categories of role, device, other thing, property, occurrence or event, and structure or organisational unit. The class characteristics describe the tangibility, sequentiality, persistence and integrity of the class.

3.5.1 User

Class Name:	User
Class Type:	Role
Class Characteristics:	Tangible; sequential; persistent; guarded
Responsibilities:	**Collaborators:**
• Store account details (name, email, password hash) • Register a new account with name, email, password and confirm password • Check that password and confirm password match • Log on and log off • Verify login credentials against the database • Maintain account status (Active / Banned) • Identify the user role after successful login	• Role • JobSeeker • Admin • ActivityLog • Notification

3.5.2 JobSeeker

Class Name:	JobSeeker
Class Type:	Role
Class Characteristics:	Tangible; sequential; persistent; guarded
Responsibilities:	**Collaborators:**
• Store job seeker profile (target industry, experience level)	• User • Resume

• Upload a resume file • Enter the target job description • Trigger the resume analysis • View ATS match score and categorised feedback • View dashboard, analysis history and score over time chart • Compare a resume and job description side by side	• JobDescription • AIAnalysisRun

3.5.3 Admin

Class Name:	Admin
Class Type:	Role
Class Characteristics:	Tangible; sequential; persistent; guarded
Responsibilities:	**Collaborators:**
• Store administrator profile (admin level, last audit date) • View system statistics such as total users, resumes analysed, average ATS score and active sessions • View a searchable and paginated list of all users • Change a user account status between Active and Banned • View system activity logs and AI processing status	• User • ActivityLog • AIAnalysisRun • Resume

3.5.4 Role

Class Name:	Role
Class Type:	Property
Class Characteristics:	Abstract; sequential; persistent; guarded
Responsibilities:	**Collaborators:**
• Store role name (Admin / Job Seeker) • Provide role description • Provide the permission set	• User

assigned to a role	

3.5.5 Resume

Class Name:	Resume
Class Type:	Other "Thing" (physical object)
Class Characteristics:	Tangible; sequential; persistent; guarded
Responsibilities:	**Collaborators:**
• Store uploaded file details (file name, type, size, path) • Validate file type and size (PDF, DOCX, TXT; maximum 5 MB) • Extract plain text from the uploaded file using the document parser • Store the extracted resume text • Render the resume preview without reloading the page	• JobSeeker • AIAnalysisRun • Document Parser Service

3.5.6 JobDescription

Class Name:	JobDescription
Class Type:	Other "Thing" (physical object)
Class Characteristics:	Tangible; sequential; persistent; guarded
Responsibilities:	**Collaborators:**
• Store the target job title and company name • Store the full job description text • Provide the job description text to the AI scoring engine	• JobSeeker • AIAnalysisRun

3.5.7 AIAnalysisRun

Class Name:	AIAnalysisRun
Class Type:	Occurrence or Event
Class Characteristics:	Abstract; sequential; persistent; guarded
Responsibilities:	**Collaborators:**
• Send the resume text and job	• Resume

description to the AI model • Generate and store the ATS match score between 0 and 100 • Track analysis status (Pending / Completed / Failed) • Record the processing time of each analysis • Link the analysis to its score metrics and feedback • Provide past analysis records for the dashboard and score chart	• JobDescription • ScoreMetrics • FeedbackSuggestion • UserRating • AI Scoring Model Service

3.5.8 ScoreMetrics

Class Name:	ScoreMetrics
Class Type:	Property
Class Characteristics:	Abstract; sequential; persistent; guarded
Responsibilities:	**Collaborators:**
• Store the category wise scores of one analysis • Hold Skills, Content Quality, Structure and Writing Style scores • Provide the score breakdown for the analytics chart	• AIAnalysisRun

3.5.9 FeedbackSuggestion

Class Name:	FeedbackSuggestion
Class Type:	Property
Class Characteristics:	Abstract; sequential; persistent; guarded
Responsibilities:	**Collaborators:**
• Store the important keywords missing from the resume • Store the identified skill gaps • Store the improvement suggestion text • Provide the missing keywords as separate recommendation tags	• AIAnalysisRun

3.5.12 ActivityLog

Class Name:	ActivityLog
Class Type:	Occurrence or Event
Class Characteristics:	Abstract; sequential; persistent; guarded
Responsibilities:	**Collaborators:**
• Record the action performed by a user • Record the IP address and the timestamp of the action • Provide the activity log list to the administrator	• User • Admin

3.6 Class Diagram

The class diagram represents the static view of the application. It identifies the major classes, their attributes and their operations, together with the associations, the aggregations and the generalisations between them. Multiplicity is shown at the ends of each association.

JobSeeker and Admin are specialisations of User, which is shown by the generalisation arrow with a hollow triangle. ScoreMetrics and FeedbackSuggestion are parts of an AIAnalysisRun and have no meaning without it, which is shown by the aggregation diamond.

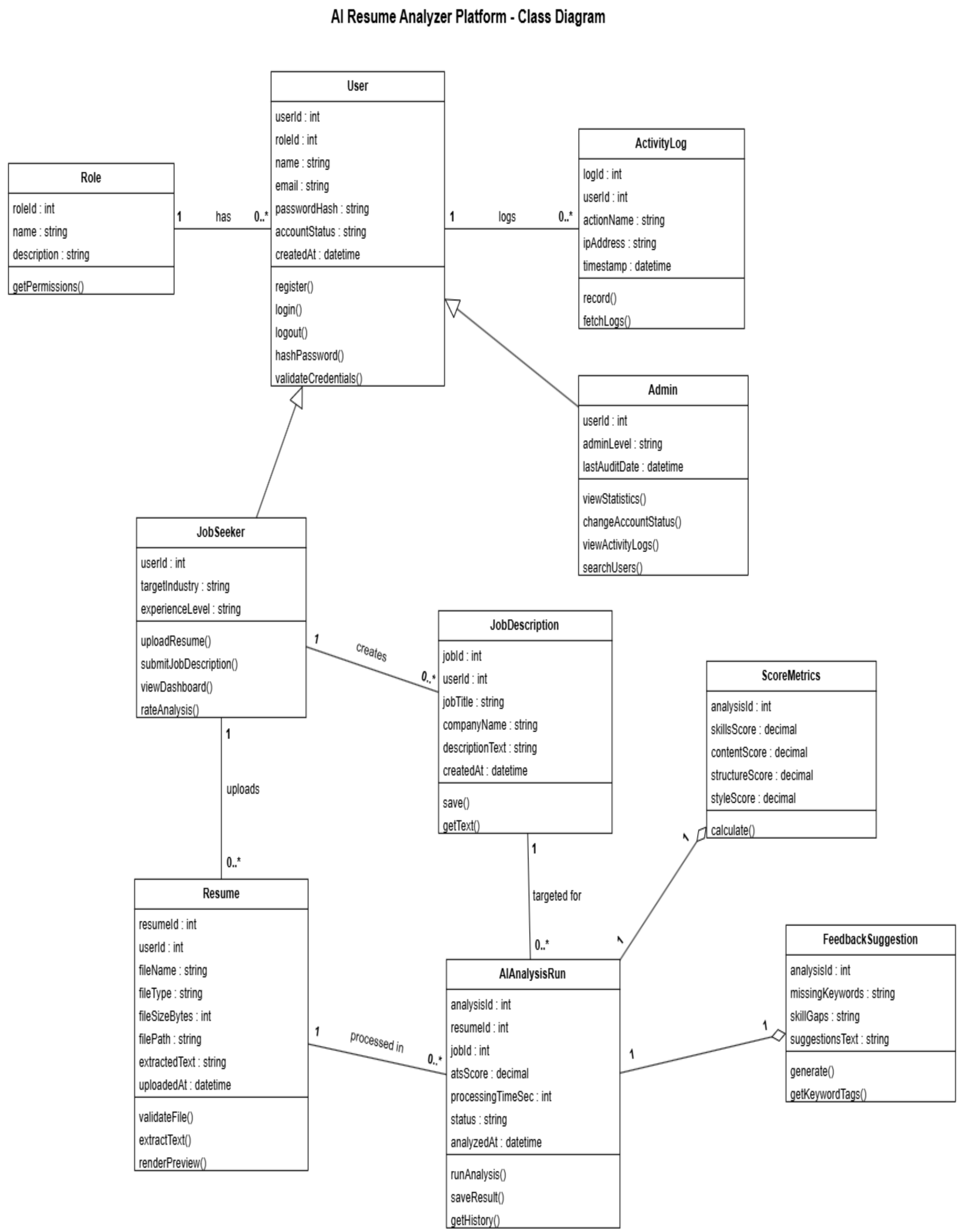

Figure 21: Class Diagram of the AI Resume Analyzer Platform

CHAPTER 4: PROJECT MANAGEMENT

Risk management ran through the whole project. The risk log was reviewed at the start of every sprint, and the process followed the four standard stages of risk identification, risk analysis, risk planning and risk monitoring.

4.1 Risk Identification

Risks were identified using the standard checklist of risk types: technology, people, organizational, tools, requirements and estimation risks.

Risk Type	ID	Possible Risk
Technology	R-01	The AI model returns text that is not valid JSON and breaks the result page.
Technology	R-02	AI analysis takes longer than the 15 second target on a loaded machine.
Technology	R-03	Text extraction fails or returns mixed text for scanned and two column resumes.
People	R-04	An access control attribute is left switched off after development testing.
People	R-05	A team member is unavailable during a sprint.
Organizational	R-06	There is no budget for a hosted cloud AI service.
Tools	R-07	Entity Framework Core migrations conflict after a model change.
Tools	R-08	The browser blocks frontend calls because the API and the client run on different ports.
Requirements	R-09	The planned scope is too large for one semester with three members.
Requirements	R-10	The accepted file types in the specification differ from the file types accepted by the code.
Estimation	R-11	The time required to integrate and stabilise the AI model is underestimated.
Estimation	R-12	The effort needed for end to end integration testing is underestimated.

Table 5: Risk identification table

4.2 Risk Analysis

Each identified risk was assessed for its probability and for the seriousness of its effect. The probability is recorded as very low (below 10%), low (10-25%), moderate (25-50%), high (50-75%) or very high (above 75%). The effect is recorded as catastrophic, which would threaten the survival of the project, serious, which would cause major delays, tolerable, where delays stay inside the allowed contingency, or insignificant.

ID	Risk	Probability	Effects
R-09	The planned scope is too large for one semester with three members.	High	Catastrophic
R-04	An access control attribute is left switched off after development testing.	Moderate	Catastrophic
R-01	The AI model returns text that is not valid JSON.	High	Serious
R-11	The time required to integrate and stabilise the AI model is underestimated.	High	Serious
R-02	AI analysis takes longer than the 15 second target.	Moderate	Serious
R-03	Text extraction fails for scanned and two column resumes.	Moderate	Serious
R-05	A team member is unavailable during a sprint.	Moderate	Serious
R-12	The effort needed for end to end integration testing is underestimated.	Moderate	Tolerable
R-08	The browser blocks frontend calls across different ports.	High	Tolerable
R-06	There is no budget for a hosted cloud AI service.	High	Tolerable
R-07	Entity Framework Core migrations conflict after a model change.	Moderate	Tolerable
R-10	Accepted file types in the specification differ from the code.	Moderate	Insignificant

Table 6: Risk analysis table

The two risks with the highest combined score, R-09 and R-04, both produced permanent changes in the project. R-09 forced the scope to be cut before any code was written, and R-04 produced the rule that a feature is not treated as finished until its access check is enabled and tested.

4.3 Risk Planning

For each risk a strategy was drawn up. An avoidance strategy reduces the probability that the risk arises, a minimisation strategy reduces the impact if it does arise, and a contingency plan describes what will be done if the risk actually occurs. The table records the action that was actually taken rather than a general intention.

ID	Strategy Type	Strategy Taken
R-01	Minimisation	The reply is cut between the first and the last brace and passed through a guard. If parsing still fails, a safe result with a clear message is returned instead of a broken page.
R-02	Minimisation	A smaller Llama 3 model was selected and a single prompt now returns

		every part of the result instead of several separate calls. A 15 second timeout with retry logic was added.
R-03	Avoidance	Only text based documents are accepted and a clear error is shown when extraction returns nothing, so a scanned file never reaches the scoring engine.
R-04	Avoidance	Every controller is reviewed before release and a feature is not treated as finished until its access check is enabled and tested.
R-05	Contingency	Work is divided by layer and the backlog is kept in one shared place so that any member can pick up an open story.
R-06	Avoidance	The system was built around a locally hosted model, which removes per request billing entirely and also became a privacy advantage.
R-07	Minimisation	Model changes are batched and each migration is reviewed and applied on a clean database before it is committed.
R-08	Avoidance	A CORS policy is registered on the server and the middleware order is fixed as CORS, authentication, then authorization, with a comment in Program.cs.
R-09	Avoidance	A schedule feasibility check was carried out before development. Job recommendation and mock interview were moved to a future release, keeping five modules complete instead of seven half finished.
R-10	Contingency	The difference was recorded during review and has been carried into the backlog for the next iteration so that the specification and the code agree.
R-11	Minimisation	A working prototype of the scoring pipeline was built during feasibility analysis, before the requirement was fixed, so the behaviour of the model was known early.
R-12	Contingency	A dedicated integration and QA sprint (Sprint 6) was reserved at the end of the schedule for end to end testing and error handling.

Table 7: Risk planning table

4.4 Risk Monitoring

Every identified risk was reassessed at the start of each sprint to decide whether it had become more or less probable and whether its effect had changed. The risk log was kept in the same shared place as the product backlog, and the highest scoring risks were discussed at the start of each sprint planning meeting.

CHAPTER 5: PROJECT PLANNING AND SCHEDULING

5.1 Function Point Estimation

Function point estimation measures the size of the software from the point of view of its functionality rather than its lines of code. Five standard function types are counted: external inputs, external outputs and external inquiries, which are the transactional functions, and internal logical files and external interface files, which are the data functions. Each item is rated simple, average or complex and given the corresponding weight.

External Inputs (EI)

Function	Complexity	Weight
Sign up form (name, email, password, confirm password)	Average	4
Login form	Simple	3
Logout request	Simple	3
Resume file upload	Complex	6
Job description entry	Average	4
Analyze Resume request	Average	4
Change account status	Simple	3
User search request	Simple	3
Subtotal		30

Table 8: External Inputs (EI) count

External Outputs (EO)

Function	Complexity	Weight
ATS match score report	Complex	7
Four category feedback report	Complex	7
Missing keyword recommendation tags	Average	5
Score over time chart	Average	5
Admin system statistics	Average	5
Subtotal		29

Table 9: External Outputs (EO) count

External Inquiries (EQ)

Function	Complexity	Weight
Resume preview	Average	4
Analysis history list	Average	4
Side by side resume and job description view	Average	4
Searchable and paginated user list	Average	4
Activity log list	Simple	3
Subtotal		19

Table 10: External Inquiries (EQ) count

Internal Logical Files (ILF)

Function	Complexity	Weight
User and Role tables	Average	10
Resume table	Average	10
Job description table	Simple	7
Analysis, score metrics and feedback tables	Complex	15
Activity log table	Simple	7
Subtotal		49

Table 11: Internal Logical Files (ILF) count

External Interface Files (EIF)

Function	Complexity	Weight
Llama 3 model reached through Ollama	Average	7
Document parser output (iTextSharp / Magick.NET)	Simple	5
Subtotal		12

Table 12: External Interface Files (EIF) count

5.1.1 Unadjusted Function Point

Function Type	Count	Weighted Total
External Inputs (EI)	9	33
External Outputs (EO)	5	29
External Inquiries (EQ)	5	19
Internal Logical Files (ILF)	5	49

External Interface Files (EIF)	2	12
Unadjusted Function Point (UFP)		139

Table 13: Unadjusted function point total

5.1.2 Value Adjustment Factor

Fourteen general system characteristics are each rated from 0, meaning no influence, to 5, meaning strong influence throughout. The sum of these ratings gives the total degree of influence, from which the value adjustment factor is calculated.

General System Characteristic	Rating (0-5)
Data communications	4
Distributed data processing	2
Performance	4
Heavily used configuration	2
Transaction rate	3
Online data entry	5
End user efficiency	4
Online update	4
Complex processing	5
Reusability	3
Installation ease	3
Operational ease	3
Multiple sites	2
Facilitate change	4
Total Degree of Influence (ΣFi)	48

Table 14: General system characteristics

$$VAF = 0.65 + \left(0.01 \times \sum Fi\right) = 0.65 \times (0.01 \times 48) = 1.13$$

$$Adjusted\ Function\ Point = UFP \times VAF = 139 \times 1.13 = 157.07 \approx 157\ FP$$

5.1.3 Effort and Duration Estimation

Taking a productivity of 21 function points per person-month, which is a reasonable figure for a web application built on established frameworks with a large amount of generated and reused code, the effort and the duration are estimated as follows.

Item	Calculation	Result
Adjusted Function Point	UFP × VAF	157 FP
Productivity	assumed	21 FP / person-month
Effort	157 / 21	7.5 person-months
Team size	actual	3 members
Duration	7.5 / 3	2.5 months (about 11 weeks)

Table 15: Effort and duration estimation

The estimate of about 11 weeks is consistent with the schedule that was actually followed, which ran for six sprints between 10 July and 28 August 2026 with a planning and analysis phase before it.

5.2 Sprint Schedule

The work was planned on a Scrum board and divided into six sprints, each mapped to one epic of the system. All six sprints were closed with every task finished: all 43 tasks on the product backlog are complete. The sprints ran between 10 July and 28 August 2026.

Sprint	Dates (2026)	Epic Worked On	Tasks	Progress
Sprint 1	10/07 to 19/07	Project Setup and Authentication	10	10 of 10 done
Sprint 2	19/07 to 26/07	Resume Upload and Processing	8	8 of 8 done
Sprint 3	28/07 to 04/08	AI Scoring and Feedback	7	7 of 7 done
Sprint 4	05/08 to 12/08	AI Scoring and Feedback, Dashboard and Job Description APIs	6	6 of 6 done
Sprint 5	20/08 to 23/08	Dashboard and UI Polish	6	6 of 6 done
Sprint 6	25/08 to 28/08	Integration QA and Demo Preparation	6	6 of 6 done

Table 16: Sprint schedule

5.3 Project Milestones

Milestone	Description
M-01	Completion of feasibility study and requirement specification
M-02	Completion of analysis modelling (use case, activity, swim lane, sequence, class)
M-03	Completion of database design and initial migrations
M-04	Completion of authentication and session management module
M-05	Completion of resume upload and text extraction module
M-06	Completion of AI scoring and feedback module

M-07	Completion of user dashboard and admin panel
M-08	Completion of integration testing and final report

Table 17: Project milestones

5.4 Project Schedule Chart

The Gantt chart below shows the real project timeline, from the start of July to early September 2026, with each activity placed on the calendar week in which it was carried out. Feasibility analysis and requirement work open the schedule, the six development sprints form the main body of it between 10 July and 28 August, and integration testing, deployment and documentation close the project. All six sprints were completed.

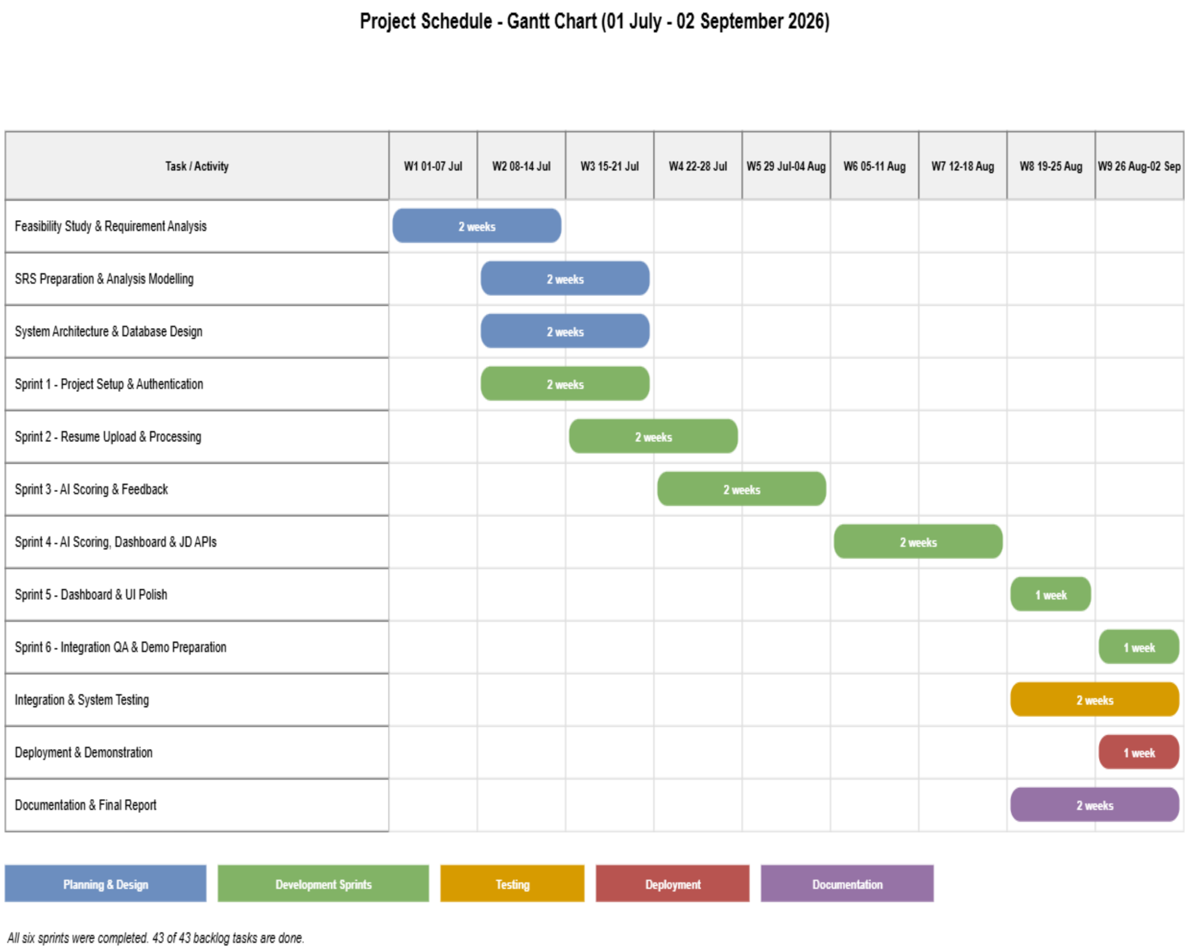

Figure 22: Project Schedule / Gantt Chart

CHAPTER 6: PROJECT ESTIMATION

The cost of the project is estimated under four headings: personnel cost, hardware cost, software cost and other cost. All amounts are in Bangladeshi Taka (TK).

6.1 Personnel Cost

6.1.1 Working Hour Calculation

Item	Value
Number of days in a year	365
Weekly holidays in a year	52
Government holidays in a year	24
Total working days (365 - (52 + 24))	289
Working days per month (289 / 12)	24.083
Working hours per day	8
Working hours per month (24.083 × 8)	192.67

Table 18: Working hour calculation

6.1.2 Salary Rates

Position	Salary / Month (TK)	Salary / Hour (TK)
System Analyst	28,000.00	145.33
Customer Communicator	15,000.00	77.85
Planner	15,000.00	77.85
Risk Analyzer	15,000.00	77.85
System Designer	20,000.00	103.81
Coder	10,000.00	51.90
Graphics Designer	10,000.00	51.90
Tester	12,000.00	62.28
Technical Communicator	8,000.00	41.52

Table 19: Personnel salary rates

6.1.3 Salary of the Technical Staff Engaged

Designation	Persons	Hours	Rate/Hr	Total Salary	First Payment (60%)	Remaining (40%)
System Analyst	1	144	145.33	20,927.34	12,556.40	8,370.93
Customer Communicator	1	48	77.85	3,737.02	2,242.21	1,494.81

Planner	1	72	77.85	5,605.54	3,363.32	2,242.21
Risk Analyzer	1	48	77.85	3,737.02	2,242.21	1,494.81
System Designer	1	120	103.81	12,456.75	7,474.05	4,982.70
Coder	3	1008	51.90	52,318.34	31,391.00	20,927.34
Graphics Designer	1	72	51.90	3,737.02	2,242.21	1,494.81
Tester	1	96	62.28	5,979.24	3,587.54	2,391.70
Technical Communicator	1	72	41.52	2,989.62	1,793.77	1,195.85
Total				111,487.89	66,892.73	44,595.16

Table 20: Personnel cost estimation

The hours above follow the schedule the project actually ran to. The analysis and design roles are engaged during the planning phase in early July, the three coders work through the seven weeks of the six development sprints between 10 July and 28 August, and testing and documentation run alongside the last two sprints. Following the standard payment method, 60% of the salary of each staff member is paid at the start of the engagement and the remaining 40% is distributed in equal weekly instalments. The total personnel cost is TK 111,487.89. This is a notional professional cost: the work was carried out by the three student members of the team, who were not paid a salary.

6.2 Hardware Cost

The team used equipment it already owned, so hardware is not charged at its full purchase price. The sum of the years digits method is used to calculate depreciation over a five year life, which gives a first year factor of 5/15, that is 33.34%. The project ran for about two months, from the start of July to the end of August 2026, so two twelfths of the first year depreciation is charged to it.

Item	Purchase Cost	Life	Year 1 Depreciation (33.34%)	Charged to Project (2 months)
Development laptop (3 units, owned by the team)	225,000.00	5 years	75,015.00	12,502.50
Printer (shared)	12,000.00	5 years	4,000.80	666.80
Router and networking device	3,500.00	5 years	1,166.90	194.48
Total				13,363.78

Table 21: Hardware cost estimation

6.3 Software Cost

Every tool used in this project is open source, free of charge, or covered by a student or community licence, and Windows was already installed on the laptops owned by the team. The software cost charged to the project is therefore zero. This was one of the conclusions of the economic feasibility study carried out before development: because Llama 3 runs locally there is no per request billing, and every other tool in the stack is free or student licensed.

Software	Cost (TK)
Windows 11 (pre-installed on team laptops)	Free / Open Source
Visual Studio 2022 Community Edition	Free / Open Source
SQL Server Developer Edition	Free / Open Source
Visual Studio Code	Free / Open Source
Ollama and Llama 3 (open source)	Free / Open Source
ASP.NET Core 8, EF Core 8, React 19, Vite, Tailwind CSS (open source)	Free / Open Source
Microsoft Office 365 Education licence	Free / Open Source
draw.io	Free / Open Source
GitHub Free plan	Free / Open Source
Total	0.00

Table 22: Software cost estimation

6.4 Other Cost

Particulars	Cost (TK)
Internet (3 members, 2 months)	7,200.00
Electricity (2 months)	4,000.00
Report printing and binding	2,500.00
Stationery and miscellaneous	1,500.00
Total	15,200.00

Table 23: Other cost estimation

6.5 Accounts Table

The accounts table brings the four headings together into the total estimated cost of the project.

Particulars	TK	TK
Personnel Cost		
System Analyst	20,927.34	
Customer Communicator	3,737.02	
Planner	5,605.54	
Risk Analyzer	3,737.02	
System Designer	12,456.75	
Coder	52,318.34	
Graphics Designer	3,737.02	
Tester	5,979.24	
Technical Communicator	2,989.62	
Subtotal - Personnel		111,487.89
Hardware Cost		
Development laptop (3 units, owned by the team)	12,502.50	
Printer (shared)	666.80	
Router and networking device	194.48	
Subtotal - Hardware		13,363.78
Software Cost		
Subtotal - Software		0.00
Other Cost		
Internet (3 members, 2 months)	7,200.00	
Electricity (2 months)	4,000.00	
Report printing and binding	2,500.00	
Stationery and miscellaneous	1,500.00	
Subtotal - Other		15,200.00
TOTAL ESTIMATED PROJECT COST		140,051.67

Table 24: Accounts table

The total estimated cost of the AI Resume Analyzer Platform is TK 140,051.67.

CHAPTER 7: DESIGNING

7.1 Data Flow Design

A data flow diagram models the events and the processes inside the system and examines how data flows into, out of and within it. Each process carries a unique number as its identifier in the top left corner and the name of the module responsible for it on the top line, with an imperative verb phrase in the main box area. External entities are drawn as ovals, data stores are drawn as open ended rectangles carrying a D reference, and every data flow arrow is named.

7.1.1 Context Level Diagram (DFD Level 0)

The context diagram shows the system as a single process with the four external entities that exchange data with it: the User or Job Seeker, the Administrator, the Document Parser Service and the AI Scoring Model Service.

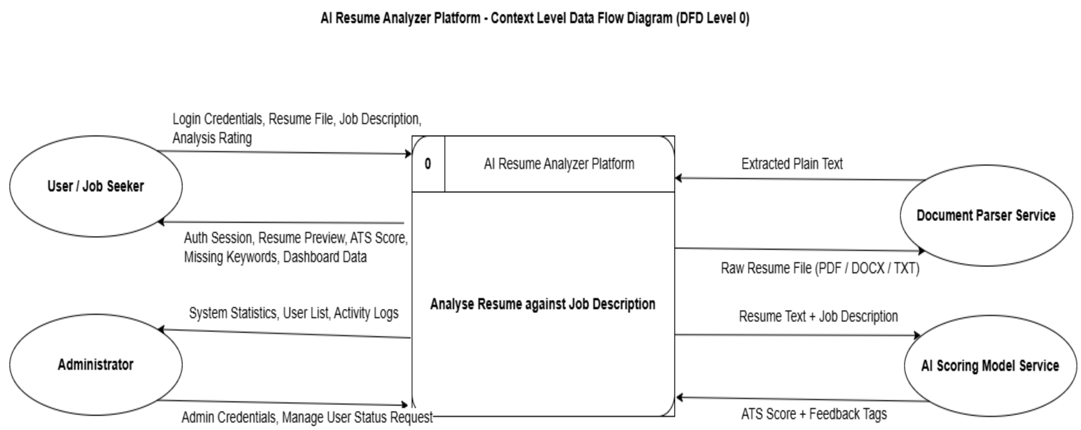

Figure 23: Context Level Data Flow Diagram of the AI Resume Analyzer Platform

7.1.2 Level 1 Data Flow Diagram

The level 1 diagram decomposes the single context process into the five modules of the system and shows the four data stores they read from and write to.

Process	Name	Responsible Module
1	Authenticate User and Create Session	Auth Module
2	Validate File and Extract Resume Text	Upload Module
3	Score Resume and Generate Feedback	AI Engine
4	Build Analysis History and Score Chart	Dashboard Module
5	Manage Users and Review System Logs	Admin Module

Table 25: Level 1 processes

Store	Contents
D1	User Accounts
D2	Extracted Resumes
D3	AI Analysis and Scores
D4	System Activity Logs

Table 26: Level 1 data stores

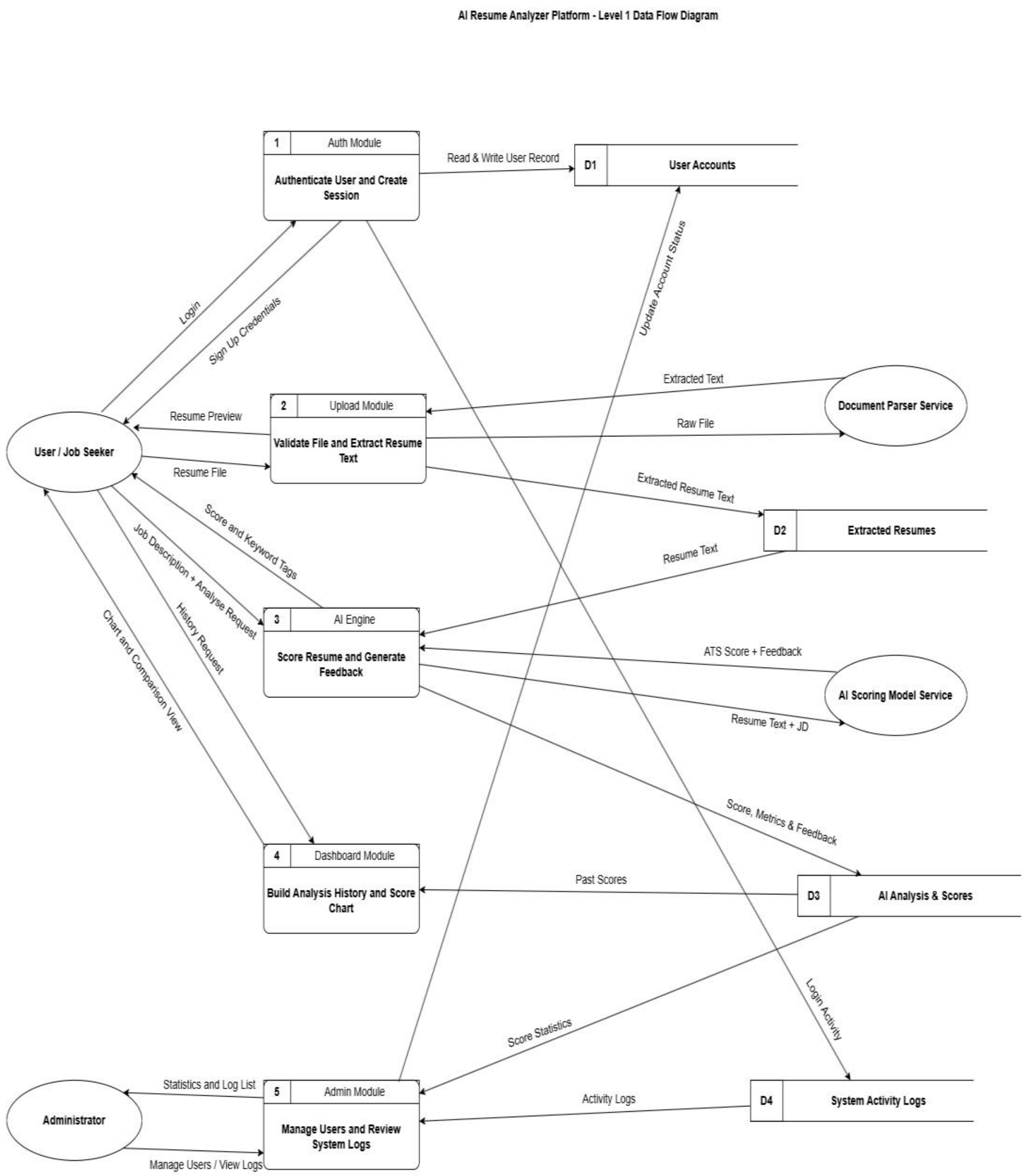

Figure 24: Level 1 Data Flow Diagram of the AI Resume Analyzer Platform

7.2 Database Design

The database provides persistent storage for the information required by the application. It is created with Entity Framework Core using code first migrations, and the schema changed five times during development, producing five migration files.

7.2.1 Entity Relationship Diagram

The entity relationship model is a detailed, logical representation of the data of the system. An entity is drawn as a rectangle, each attribute is drawn as an ellipse joined to its entity, a key attribute is underlined, and a relationship is drawn as a diamond with the multiplicity written on the connecting lines.

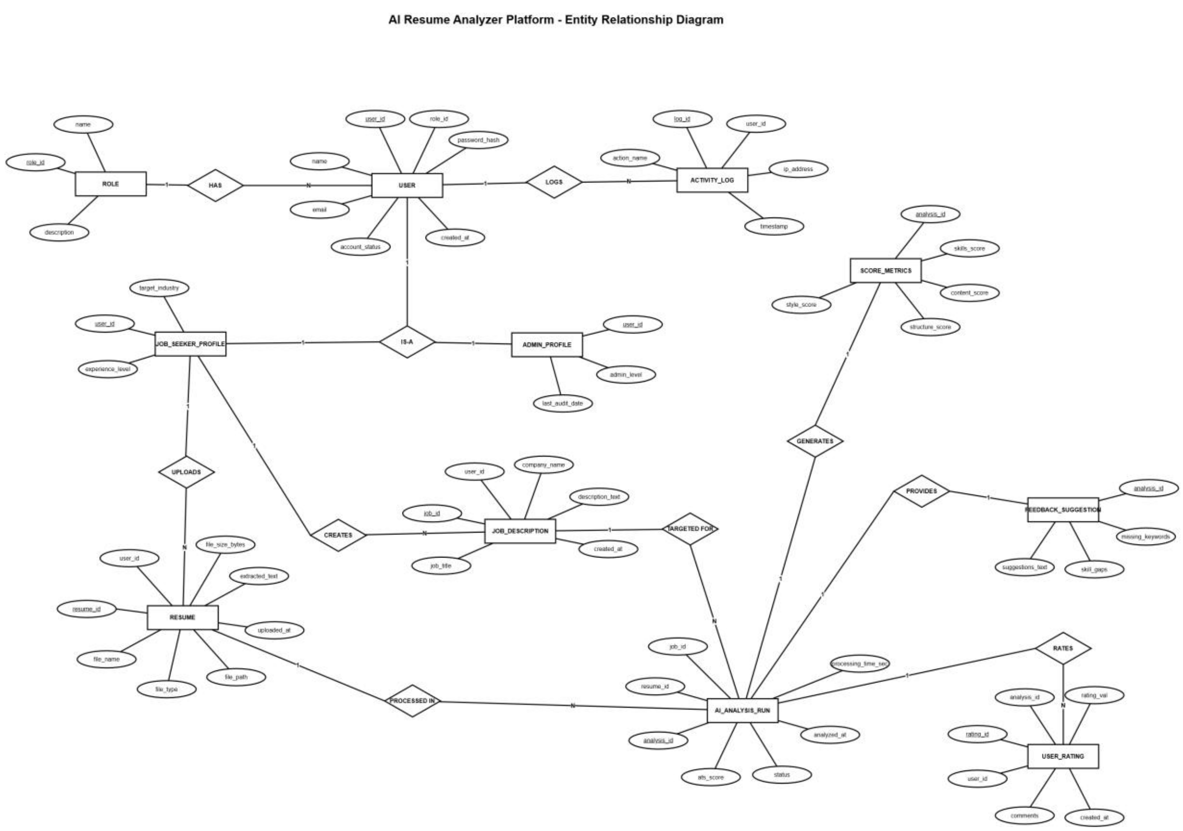

Figure 25: Entity Relationship Diagram of the AI Resume Analyzer Platform

7.2.2 Entity Description

Entity	Description
ROLE	Holds the role names Admin and Job Seeker together with a description.
USER	Holds the account details of every registered person, including the password hash and the account status.
JOB_SEEKER_PROFILE	Specialisation of USER holding the target industry and the experience level.
ADMIN_PROFILE	Specialisation of USER holding the administrative level and the last audit date.
RESUME	Holds the uploaded file details, the file path and the text extracted from the

	resume.
JOB_DESCRIPTION	Holds the target job title, the company name and the full job description text.
AI_ANALYSIS_RUN	One execution of the scoring engine, holding the ATS score, the processing time and the status.
SCORE_METRICS	The four category scores produced by one analysis.
FEEDBACK_SUGGESTION	The missing keywords, the skill gaps and the written suggestions of one analysis.
USER_RATING	The rating and comment a user submits for a completed analysis.
ACTIVITY_LOG	A record of every significant action, with the IP address and the timestamp.

Table 27: Entity description

7.3 Interface Design

The interface design describes the user facing components of the system. The application uses a consistent dark theme, and it provides separate interfaces for the Job Seeker and for the Administrator. The screens below show the main interfaces of the running application.

7.3.1 Login Interface

The login interface lets a registered user or an administrator authenticate with an email address and a password. The left panel carries the product identity and the right panel holds the credential form, with a link for a new user to create an account.

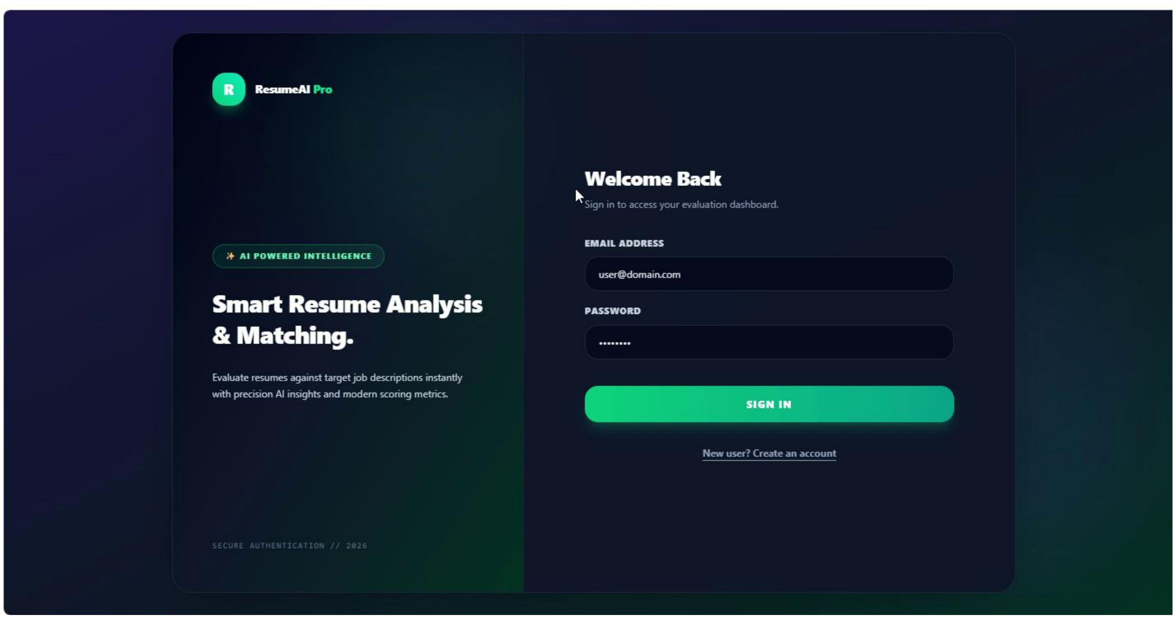

Figure 26: Login Interface

7.3.2 Sign Up Interface

The sign up interface collects the name, the email address, the password and the confirm password. The system checks that the two passwords match and reports the error without reloading the page.

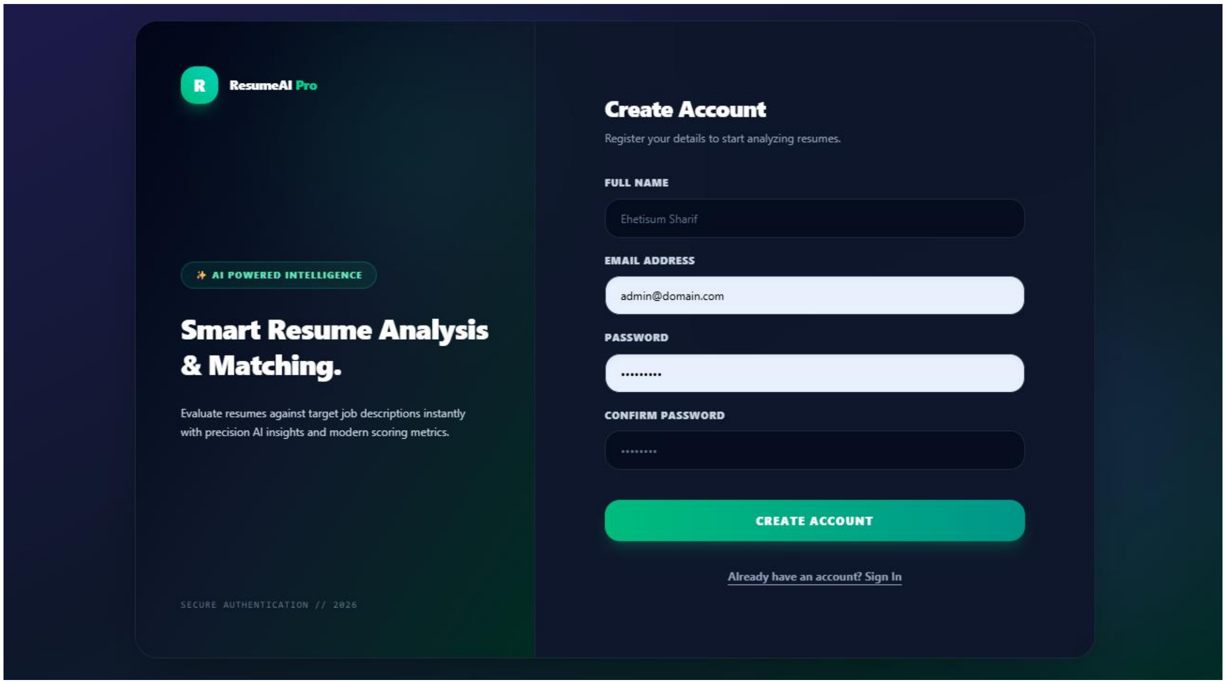

Figure 27: Sign Up Interface

7.3.3 Resume Upload, Preview and Job Description Interface

The upload panel accepts a resume by drag and drop or through a file selector and shows the accepted file with its name and size. The first page of the resume is rendered directly below it, the target job description is entered underneath, and the button reports the running state while the analysis is in progress. The right panel waits for the diagnostic result.

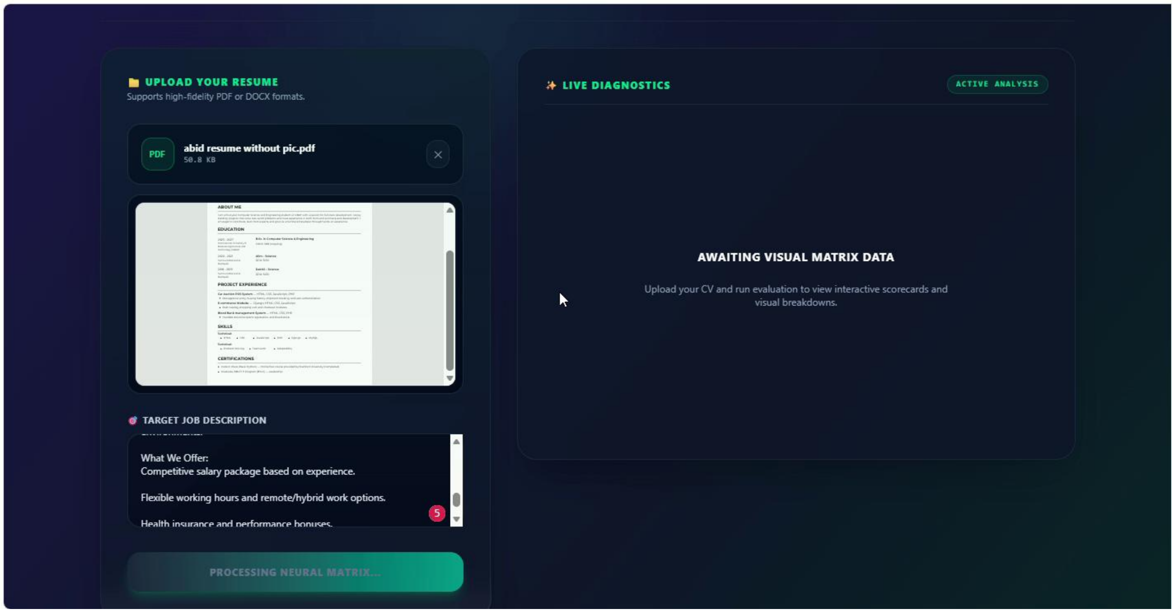

Figure 28: Resume Upload, Preview and Job Description Interface

7.3.4 Analysis Result Interface

The result panel shows the ATS compatibility index as an overall match score, a short AI summary of the parse, and the skills and keywords that were found in the resume, followed by the numbered improvement suggestions.

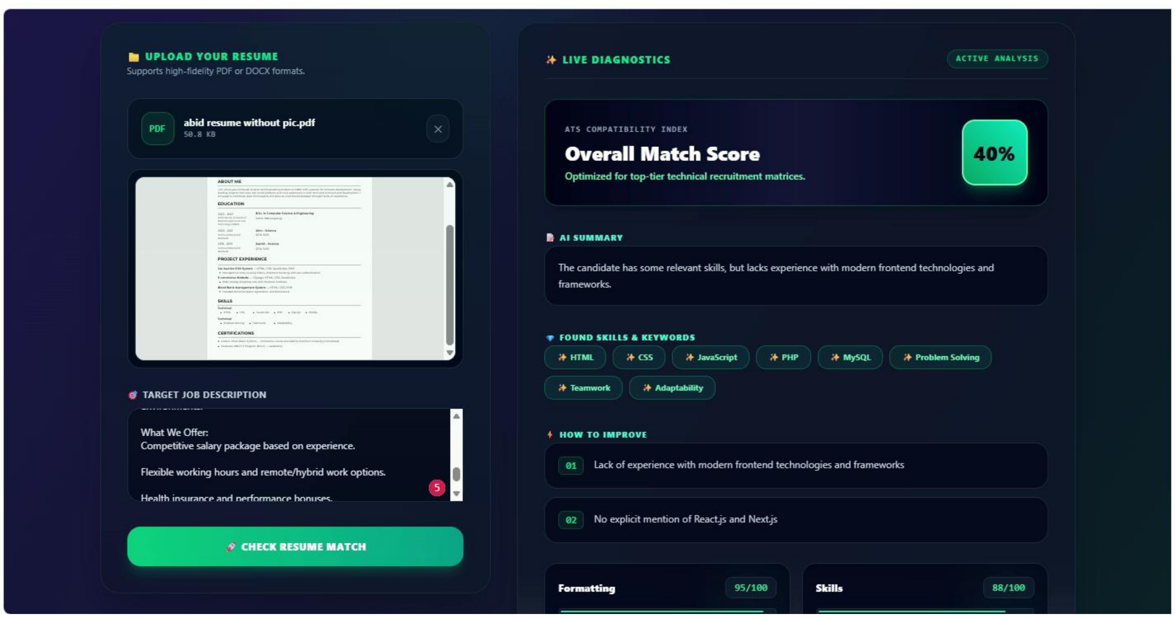

Figure 29: Analysis Result Interface

7.3.5 Category Scores and Missing Keyword Interface

The feedback is broken down into the scored categories, each with its own value out of 100 and a short comment. Below them the missing skills and requirements are listed as separate recommendation tags so the user knows exactly which keywords to add.

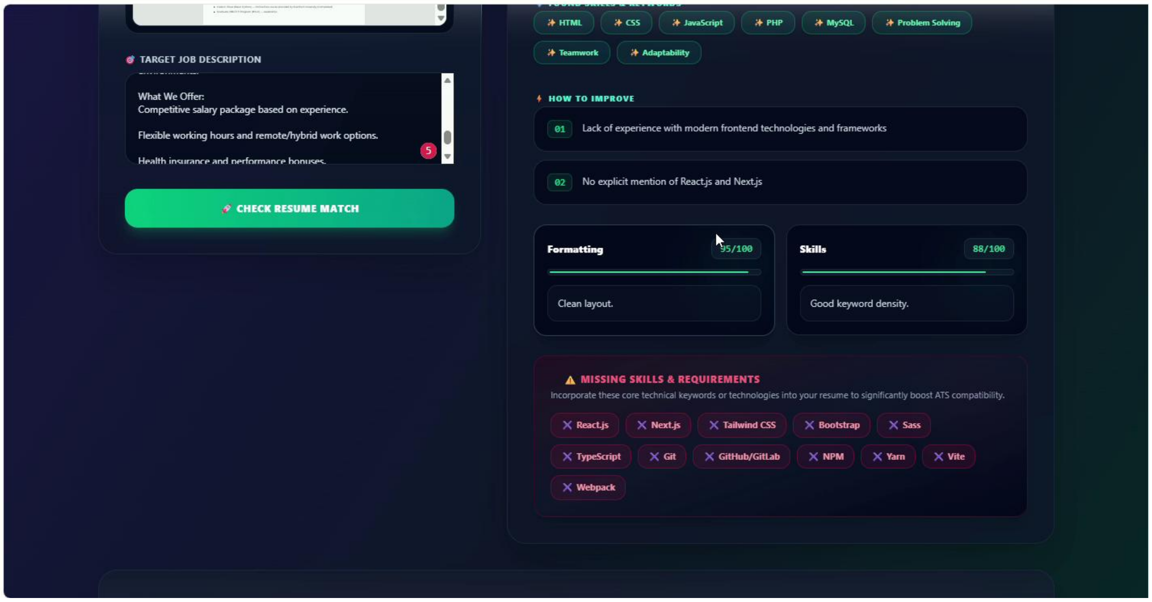

Figure 30: Category Scores and Missing Keyword Interface

7.3.6 User Dashboard Interface

The dashboard tracks progress across analyses. It holds the score history graph, the before and after comparison of the previous and the updated resume, and the breakdown of which missing skills have since been resolved.

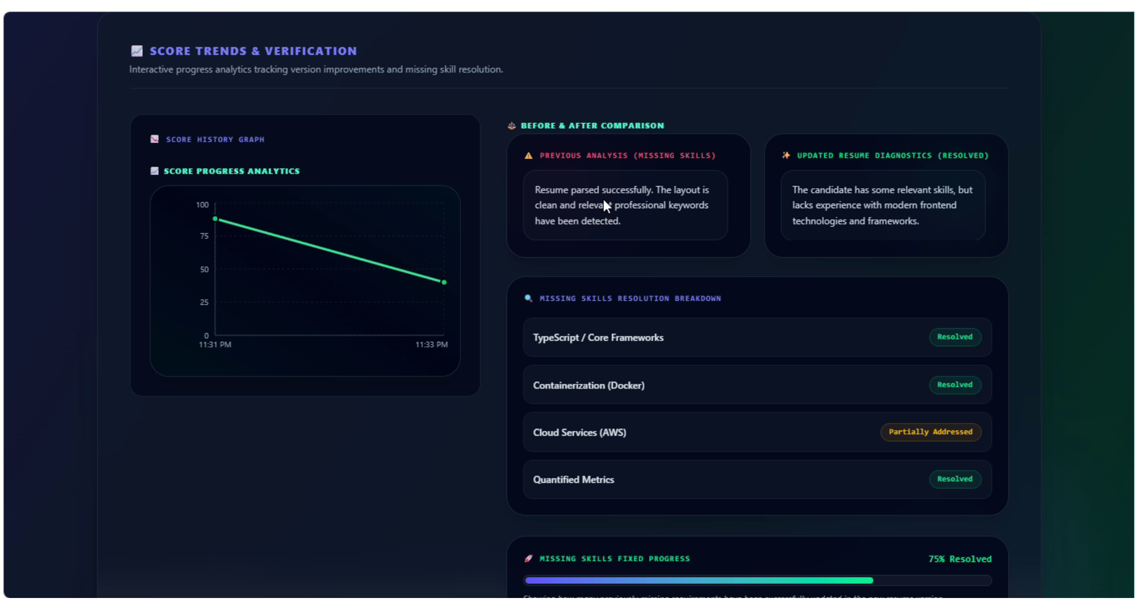

Figure 31: User Dashboard Interface

7.3.7 Missing Skill Resolution Interface

Each previously missing requirement is listed with its current state, and the progress bar summarises how many of them have been resolved in the newer version of the resume.

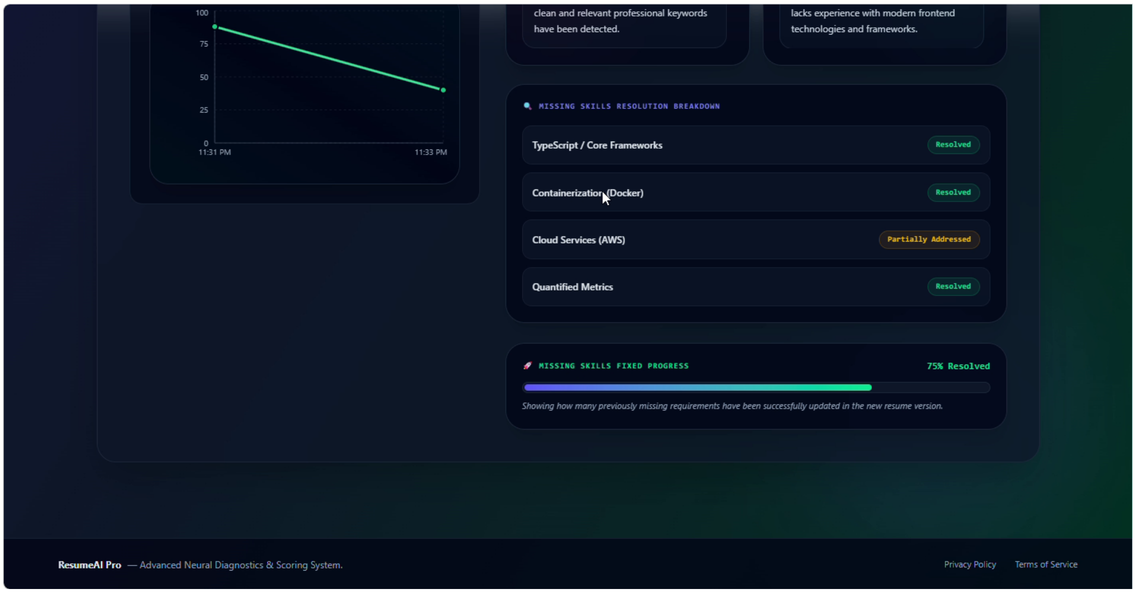

Figure 32: Missing Skill Resolution Interface

7.3.8 Admin Dashboard and User Management Interface

The administrator dashboard shows the total registered users, the total resumes analysed, the average ATS score and the number of active user sessions. Below the statistics, the user list is searchable and paginated, each row showing the role and the account state with a control to suspend or restore the account.

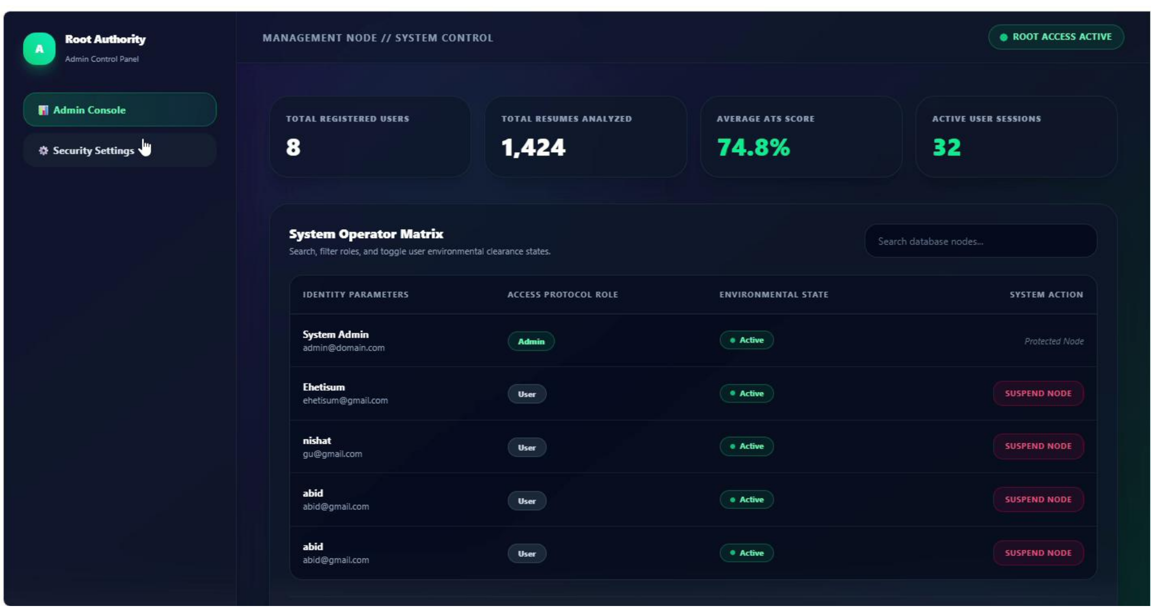

Figure 33: Admin Dashboard and User Management Interface

7.3.9 Admin Activity Log Interface

The system telemetry log records timestamped entries for system events such as model initialisation and secure session establishment, which lets the administrator monitor how the platform is behaving.

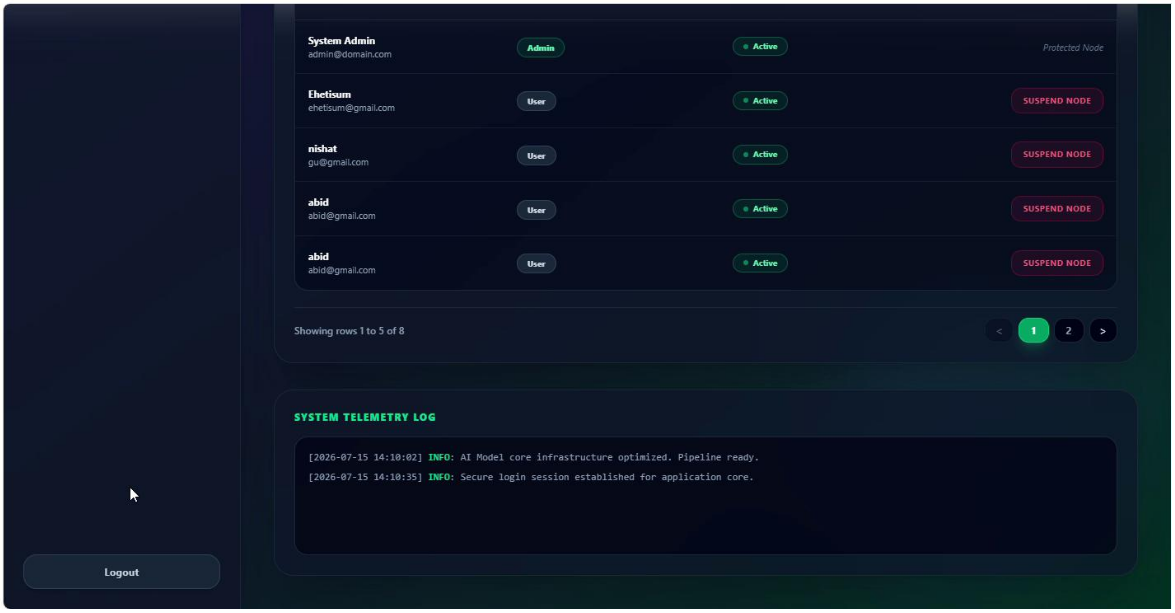

Figure 34: Admin Activity Log Interface

7.3.10 Admin Security Settings Interface

The security settings screen lets the administrator change the root authorisation email address and the primary access password, with the password repeated for confirmation.

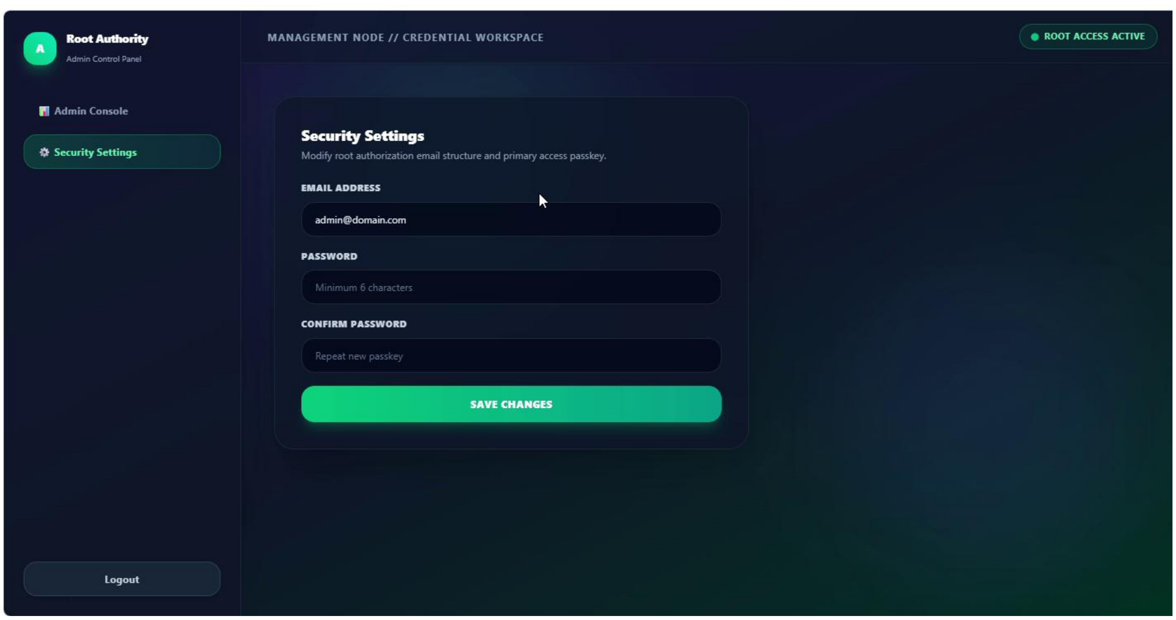

Figure 35: Admin Security Settings Interface

CHAPTER 8: TESTING

8.1 Introduction

Testing is the process of evaluating a system or its components with the intention of finding whether it satisfies the specified requirements. The purpose of testing in this project is to identify errors, to validate the functionality of the system, to confirm that the requirements defined in Chapter 2 have been satisfied, and to improve the reliability of the final software. Each major requirement is associated with one or more test cases.

8.2 Testing Strategy

8.2.1 Unit Testing

Individual units of source code, such as the file validator and the JSON guard in the scoring service, were tested by the developer responsible for them before the work was handed over.

8.2.2 Integration Testing

The combined behaviour of the React client, the Web API and the local AI model was tested, since the three parts run as separate processes and communicate over HTTP.

8.2.3 Functional Testing

Black box testing based on the specification. Each functional requirement was exercised through the user interface and the result compared with the expected behaviour.

8.2.4 System Testing

The complete application was tested as a whole once every module was integrated, following the full path from upload to score to feedback.

8.2.5 Regression Testing

After every bug fix the surrounding features were retested, because a change in the scoring service or in a shared component can affect other screens.

8.2.6 User Interface Testing

The screens were checked for correct display and navigation, and responsive behaviour was verified on mobile, tablet and desktop widths.

8.2.7 Security Testing

Authentication, authorization, session handling and protection against unauthorised access were tested, including direct calls to the API without a token.

8.2.8 Database Testing

Insert, update, retrieve and delete operations were verified through Entity Framework Core, along with the relationships between users, resumes and analyses.

8.2.9 Performance Testing

The time taken for preview and text extraction and for a full AI analysis was measured against the targets stated in NFR-1.

8.3 Produced Test Cases

The following test cases cover the functional requirements and the measurable non-functional requirements identified for the system.

Test ID	Requirement	Test Case	Expected Result	Status
TC-01	FR-01	Register with valid name, email, password and confirm password	Account is created and a registration successful message is shown	Pass
TC-02	FR-01	Register with an email address that already exists	An "Email already registered" message is shown	Pass
TC-03	FR-02	Register with two passwords that do not match	"Passwords do not match" is shown without the page reloading	Pass
TC-04	FR-03	Log in with valid user credentials	The Resume Analysis Dashboard is displayed	Pass
TC-05	FR-03	Log in with valid administrator credentials	The Admin Dashboard is displayed	Pass
TC-06	FR-04	Log in with a wrong password	"Invalid email or password" is displayed	Pass
TC-07	FR-04	Log in with an unregistered email address	"Invalid email or password" is displayed	Pass
TC-08	FR-05	Inspect the issued token after a successful login	The token carries the correct user role	Pass
TC-09	FR-06	Click Logout	The session ends and the login page is displayed	Pass
TC-10	FR-07	Upload a resume using drag and drop	The file is accepted and passed to validation	Pass
TC-11	FR-07	Upload a resume using the file selector	The file is accepted and passed to validation	Pass
TC-12	FR-08	Upload a valid PDF resume smaller than 5 MB	The file is accepted	Pass
TC-13	FR-08	Upload a file larger than 5 MB	A file validation error is displayed immediately	Pass
TC-14	FR-08	Upload an unsupported file type such as .png	A file validation error is displayed immediately	Pass
TC-15	FR-09	Upload a valid resume and observe the page	A preview of the resume is shown without a page reload	Pass

TC-16	FR-10	Upload a text based PDF resume	The plain text is extracted and stored	Pass
TC-17	FR-10	Upload a scanned PDF resume	A clear "could not read resume" error is shown	Pass
TC-18	FR-11	Enter a job title, company name and job description	The job description is stored against the user	Pass
TC-19	FR-12	Click Analyze Resume with a resume and a job description present	The resume text and the job description are sent to the AI model	Pass
TC-20	FR-13	Observe the screen while the analysis is running	An "Analyzing Resume..." message is displayed	Pass
TC-21	FR-14	Complete a successful analysis	An ATS match score between 0% and 100% is displayed	Pass
TC-22	FR-15	Review the feedback panel after an analysis	Feedback is grouped into Skills, Content Quality, Structure and Writing Style	Pass
TC-23	FR-16	Analyse a resume that is missing skills named in the job description	The missing skills are identified	Pass
TC-24	FR-17	Review the result screen	Missing keywords are displayed as separate recommendation tags	Pass
TC-25	FR-18	Try to submit a rating before an analysis has completed	The rating control is not available	Pass
TC-26	FR-18	Submit a rating and a comment after a successful analysis	The rating is stored against the analysis	Pass
TC-27	FR-19	Force the AI model to return an invalid reply	An honest error message is shown instead of a score	Pass
TC-28	FR-20	Open the user dashboard with previous analyses present	All past resumes, ATS scores and dates are listed	Pass
TC-29	FR-20	Open the user dashboard with no previous analysis	A "no analysis yet" message is displayed	Pass
TC-30	FR-21	Open the dashboard after several analyses	A Score Over Time chart is displayed	Pass
TC-31	FR-22	Select a previous analysis from the dashboard	The resume and the job description are shown side by side	Pass
TC-32	FR-23	Log in as an administrator	A separate Admin Dashboard is displayed	Pass
TC-33	FR-24	Open the Admin Dashboard	Total users, total resumes analysed, average ATS score and active sessions are displayed	Pass
TC-34	FR-25	Search the user list by name or email	Matching user records are displayed with role information	Pass
TC-35	FR-25	Move to the next page of the user list	The next set of user records is displayed	Pass
TC-36	FR-26	Change a user account status from Active to Banned	The change is saved and the refreshed list shows the new status	Pass

TC-37	FR-27	Open the activity log page as an administrator	System activity, AI processing status and error logs are listed	Pass
TC-38	NFR-2	Call a protected API endpoint without a token	The request is rejected with an unauthorised response	Pass
TC-39	NFR-2	Call an administrator endpoint using a normal user token	Access is denied	Pass
TC-40	NFR-2	Inspect the stored password of a registered user	Only the password hash is stored, never the plain password	Pass
TC-41	NFR-1	Measure the time taken for preview and text extraction	Extraction and preview complete within 5 seconds	Pass
TC-42	NFR-1	Measure the time taken for a full AI analysis on an idle machine	The analysis completes within 15 seconds	Pass
TC-43	NFR-3	Upload the same resume twice and compare the extracted text	The extracted text is identical on both runs	Pass
TC-44	NFR-8	Delete nothing and re-open a past analysis	The analysis is still linked to the correct user and resume	Pass

Table 28: Produced test cases

A total of 44 test cases were produced and executed against the functional and the measurable non-functional requirements. All 44 test cases passed, which confirms that every implemented module behaves according to the specification. All six sprints, including the final integration QA sprint, were closed with every task finished.

CHAPTER 9: CONCLUSION

9.1 Brief Overview of the Project

The AI Resume Analyzer Platform is a web application that gives a job seeker fast, private and free feedback on a resume before it is submitted. A user uploads a resume, pastes the description of the job they are applying for, and the system returns an ATS match score between 0% and 100%, feedback grouped into Skills, Content Quality, Structure and Writing Style, and the list of important keywords that the resume is missing. Every analysis is stored, so the dashboard can show a Score Over Time chart and a side by side comparison of any earlier resume and job description.

The system is organised into five modules: authentication and session management, resume upload and text extraction, AI scoring and feedback, user dashboard and analytics, and admin panel management. It follows a layered architecture with a separate React client, an ASP.NET Core 8 Web API, and SQL Server reached only through Entity Framework Core. The analysis is performed by Llama 3 running locally through Ollama, which keeps every resume on the machine and removes any per request cost.

The project was developed by three members using Scrum across six sprints between July and August 2026. Feasibility analysis was carried out before the requirements were fixed, and it changed the project in four concrete ways: a local model was chosen instead of a cloud service, a failed analysis was specified to show an honest error rather than a score, accepted files were limited to text based documents, and two planned features were deferred so that five modules could be finished properly. The estimated size of the system is about 168 function points and the estimated cost is TK 140,051.67.

The analysis model was documented with a use case diagram and use case text, five activity diagrams, six swim lane diagrams, six sequence diagrams, a CRC model of twelve classes and a class diagram. The design was documented with a context level and a level 1 data flow diagram and an entity relationship diagram. Forty four test cases were produced against the functional and non-functional requirements, and all of them passed.

9.2 Limitations of the Project

The following limitations are known and are recorded honestly so that they can be addressed in a future release.

9.2.1

Scanned resumes are not supported. A scanned PDF is really an image, so the text extractor returns nothing and the system shows an error instead of a score. Optical character recognition was left out because it did not fit the available time.

9.2.2

Two column resumes can produce mixed up text. The extractor reads the page in a single flow, so a two column layout is sometimes returned in the wrong order, which lowers the quality of the analysis.

9.2.3

The specification allows PDF, DOCX and TXT files, but the upload endpoint currently accepts only PDF and DOCX. This difference was found during review and has been carried to the backlog for the next iteration.

9.2.4

The speed of the analysis depends on the load of the machine. Because Llama 3 runs locally, a busy machine can push a single analysis past the 15 second target.

9.2.5

The AI model does not always reply in a clean format. A guard cleans the reply and falls back to a safe result, but an occasional analysis still has to be retried.

9.2.6

Category scores, missing keywords and detailed feedback are stored as JSON strings inside one table. This was quick to build, but those fields cannot be searched with an ordinary SQL query.

9.2.7

The platform runs on a single local server. Multiple sites, load balancing and horizontal scaling were not part of this release.

9.2.8

Job recommendation and mock interview were planned at the start but were removed during the schedule feasibility check, so they are not part of this version.

9.2.9

The system has been tested by the project team only. A wider usability study with real job seekers has not yet been carried out.

9.3 Future Work

- Add optical character recognition so that scanned resumes can be analysed.
- Improve text extraction for two column and heavily formatted resumes.
- Align the accepted file types in the code with the specification by adding TXT support.
- Move the category scores and keywords out of JSON strings into their own tables so that they can be queried with SQL.
- Add the job recommendation and mock interview modules that were deferred from this release.
- Move the AI model to a dedicated server so that analysis time does not depend on the load of the development machine.
- Carry out a usability study with real job seekers outside the project team.

REFERENCES

[1] Microsoft. ASP.NET Core Documentation. Microsoft Learn. learn.microsoft.com/aspnet/core

[2] Microsoft. Entity Framework Core Documentation. Microsoft Learn. learn.microsoft.com/ef/core

[3] Microsoft. Semantic Kernel Documentation. Microsoft Learn. learn.microsoft.com/semantic-kernel

[4] Meta AI (2024). Llama 3 Model Documentation. llama.meta.com

[5] Ollama. Ollama Documentation. ollama.com

[6] React. React 19 Documentation. react.dev

[7] Project repository: github.com/EhetisumSharif/ai-resume-analyzer